

UNIVERSIDAD DE LA REPÚBLICA URUGUAY



FACULTAD DE INGENIERÍA

Diseño y desarrollo de un robot educativo con procesamiento de imágenes

Informe de Proyecto de Grado presentado por

Facundo Cardozo, Joaquin Sanson, Luis Zamora y Nadia Verdier

en cumplimiento parcial de los requerimientos para la graduación de la carrera de Ingeniería en Computación de Facultad de Ingeniería de la Universidad de la República

Supervisores

Jorge Visca
Ewelina Bakala

Montevideo, 26 de octubre de 2026



Diseño y desarrollo de un robot educativo con procesamiento de imágenes por Facundo Cardozo, Joaquin Sanson, Luis Zamora y Nadia Verdier tiene licencia CC Atribución 4.0.

Agradecimientos

Agradecemos especialmente a las maestras Elisa, Cecilia y María José por su valiosa colaboración durante la etapa de validación del proyecto en contexto real de aula. Su disposición para permitir la realización de pruebas con Robotito en sus clases fue fundamental para evaluar la propuesta en condiciones auténticas de uso educativo.

Asimismo, agradecemos el tiempo dedicado a responder nuestras consultas y a compartir observaciones didácticas y operativas a partir de su experiencia docente. Sus aportes y retroalimentación contribuyeron de forma directa a mejorar tanto el diseño de las actividades como decisiones técnicas del sistema, fortaleciendo la calidad y pertinencia de este trabajo.

Resumen

Robotito es una plataforma educativa de software libre y hardware abierto utilizada para introducir conceptos de robótica y pensamiento computacional en niños de nivel inicial y primeros años de primaria. En su versión original, el robot cuenta con sensores básicos de color y de distancia que limitan el tipo de actividades que se le pueden proponer a los estudiantes.

Este trabajo presenta el diseño e implementación de una extensión de Robotito que incorpora capacidades de procesamiento de imágenes mediante una cámara HuskyLens, ampliando así su capacidad de percibir el entorno. La solución integra tres componentes principales: un módulo que permite la comunicación entre la cámara HuskyLens y la placa ESP32 del robot y permite el seguimiento de líneas y la lectura de marcadores visuales (AprilTags); un sistema de control del comportamiento basado en Árboles de Comportamiento (*Behavior Trees*), implementado como una biblioteca reutilizable en Lua; y un configurador web que permite a los docentes definir secuencias de actividad y transferirlas al robot de forma inalámbrica mediante comunicación Bluetooth Low Energy (BLE), con persistencia de datos en la memoria no volátil del ESP32.

A partir de estas capacidades se diseñó una actividad educativa en la que Robotito recorre un circuito siguiendo una línea, se detiene en distintas estaciones marcadas con tags y valida si el orden de las tarjetas ubicadas por los estudiantes corresponde a una secuencia previamente configurada por el docente, brindando retroalimentación visual inmediata mediante luces LED.

La propuesta fue validada mediante entrevistas con docentes de educación inicial y primaria, y a través de dos sesiones prácticas con estudiantes de distintas edades en contextos reales de aula. Los resultados obtenidos muestran que la solución es técnicamente viable, resulta comprensible y motivadora para los niños, y permite integrar contenidos curriculares diversos. A partir del *feedback* recibido se realizaron ajustes al diseño original, entre ellos la incorporación de funciones de pausa y reinicio que otorgan a los docentes mayor control sobre el flujo de la actividad en el aula.

Este trabajo constituye un aporte concreto a la plataforma Robotito, sentando las bases para futuras extensiones que aprovechen la percepción visual y la arquitectura modular desarrollada.

Palabras clave: Robótica educativa, procesamiento de imágenes, Robotito, ESP32, Husky-Lens, I2C, Lua RTOS, C++, Árboles de comportamiento, Bluetooth Low Energy, NVS

Índice general

Resumen v

1. Introducción 1
- 1.1. Actividad didáctica . . . 2
- 1.2. Objetivos . . . 3
- 1.3. Organización del documento . . . 4

2. Revisión de antecedentes 7
- 2.1. Robotito . . . 7
  - 2.1.1. Actividades . . . 7
- 2.2. Robots educativos para actividades secuenciales . . . 8

3. Análisis del problema y diseño de la solución 9
- 3.1. Análisis del problema . . . 9
  - 3.1.1. Problemas identificados . . . 9
  - 3.1.2. Requerimientos de la solución . . . 10
- 3.2. Diseño de la solución . . . 11
  - 3.2.1. Módulos a implementar . . . 11
- 3.3. Diseño de la actividad . . . 11
  - 3.3.1. Caso de uso . . . 12
  - 3.3.2. Desarrollo de la actividad . . . 14
  - 3.3.3. Ejecución del robot . . . 16

4. Implementación de la solución 19
- 4.1. Módulo de percepción visual con HuskyLens . . . 19
  - 4.1.1. Funcionamiento de HuskyLens . . . 19
  - 4.1.2. Integración con Robotito . . . 21
- 4.2. Control del comportamiento con Behavior Trees . . . 23
  - 4.2.1. Fundamentos teóricos . . . 24
  - 4.2.2. Arquitectura del módulo BT . . . 25
  - 4.2.3. Implementación de nodos . . . 25
- 4.3. Configurador web para la preparación de secuencias . . . 28
  - 4.3.1. Rol del configurador web . . . 28
  - 4.3.2. Definición de tags y estaciones . . . 28

4.3.3. Construcción y validación de secuencias 30
4.3.4. Biblioteca de actividades y flujo de uso 30
4.4. Configuración de actividades y comunicación BLE 32
4.4.1. Gestión de Datos y Persistencia 32
4.4.2. Arquitectura Multitarea y Sincronización 32
4.4.3. Integridad de Datos y Mutex 32
4.4.4. Interfaz con la Capa de Aplicación (API Lua) 33

5. Experimentación **35**
5.1. Metodología 35
5.1.1. Entrevistas con docentes 35
5.1.2. Primera sesión práctica con estudiantes 36
5.1.3. Segunda sesión práctica con estudiantes 40
5.2. Resultados 43
5.2.1. Retroalimentación de las entrevistas docentes 43
5.2.2. Observaciones de la primera sesión con estudiantes 46
5.2.3. Observaciones de la segunda sesión con estudiantes 47
5.3. Análisis, discusión y ajustes 49
5.3.1. Fortalezas identificadas 49
5.3.2. Desafíos y áreas de mejora 50
5.3.3. Validación de decisiones de diseño 51
5.3.4. Ajustes realizados basados en el feedback 51
5.4. Conclusiones del proceso de experimentación 52

6. Conclusiones y Trabajo Futuro **55**
6.1. Conclusiones 55
6.2. Líneas de trabajo futuro 56

Referencias **59**

A. Tabla comparativa de cámaras **61**

B. Modelado de carcasa para Robotito **63**
B.1. Desarrollo del diseño de la carcasa 63
B.1.1. Diseño conceptual 65
B.1.2. Proceso de modelado CAD 65
B.1.3. Resultados y mejoras futuras 66

Capítulo 1

Introducción

Robotito es una plataforma educativa de software libre y hardware abierto pensada para actividades de robótica y programación en las primeras etapas de la formación (Tejera y cols., 2019). Si bien el robot ya tiene capacidad de uso en el aula, tiene una limitación clara: cuenta con sensores que permiten un reconocimiento acotado de su entorno, limitando el tipo de actividades que se le puede proponer a los estudiantes.

A partir de esta carencia, se propone como punto de partida de este trabajo integrar una herramienta de procesamiento de imágenes al robot con el objetivo de extender su capacidad de percibir ciertos elementos de su entorno y reaccionar adecuadamente. Tomando esa propuesta, la tarea se dividió en dos partes: integrar una cámara a Robotito y, una vez disponible la nueva capacidad de percepción visual, diseñar y ejecutar una actividad educativa que la aproveche.

Para resolver esto se incorporaron los siguientes componentes a Robotito: una cámara HuskyLens (módulo de procesamiento de imágenes capaz de seguir líneas y reconocer tags sin sobrecargar al microcontrolador del robot), un sistema de control basado en árboles de comportamiento que organiza y coordina las acciones del robot, una aplicación web donde el docente configura la actividad, y un canal BLE (*Bluetooth Low Energy*, una variante de Bluetooth pensada para enviar poca cantidad de datos consumiendo muy poca energía) que transporta esa configuración hacia el robot. Con estos componentes funcionando en conjunto, se logra plantear una actividad donde Robotito puede realizar un recorrido siguiendo una línea continua, reconocer estaciones (puntos de parada del recorrido) marcadas con tags (de tipo *AprilTag*) y verificar si una secuencia armada por los estudiantes es correcta. El motivo de cada una de estas elecciones técnicas se explica con más detalle en los capítulos siguientes.

Para facilitar la comprensión de las secciones siguientes, se presenta una breve descripción de la actividad desarrollada y los objetivos de la solución. Esto permite establecer desde el comienzo el contexto de uso de los nuevos componentes incorporados a Robotito, así como las capacidades y limitaciones consideradas en el proyecto.

1.1. Actividad didáctica

La actividad desarrollada consiste en lograr que el robot realice un recorrido guiado por una línea y verifique, en sus estaciones de parada, si las tarjetas ubicadas por los estudiantes respetan una secuencia previamente configurada por el docente. Su objetivo es estimular la habilidad de secuenciación —entendida como la capacidad de ordenar elementos o acciones siguiendo una relación lógica o temporal— además de acercar a los estudiantes a conceptos de robótica y pensamiento computacional.

El docente elige el contenido y el orden de la secuencia que se va a trabajar, por ejemplo, los pasos de una rutina diaria o los episodios de un cuento. Cada paso se representa mediante una tarjeta que contiene un marcador visual de tipo *AprilTag* (en adelante, tag) y un espacio libre para un dibujo, una imagen o un objeto. El docente se asegura de que cada representación esté asociada a la tarjeta cuyo tag ocupa la posición correspondiente en la secuencia esperada; luego configura esa lista ordenada de tags en el configurador web y la transfiere al robot mediante BLE. También prepara el tablero modular, con una línea continua entre el inicio y el fin y estaciones señalizadas con tags de parada, presenta la consigna y acompaña el trabajo del grupo.

Los estudiantes representan los pasos de la secuencia con dibujos, imágenes u objetos, acuerdan su orden y ubican las tarjetas en los espacios destinados a ellas junto a las estaciones del tablero. Por ejemplo, para una rutina de lavado de manos pueden ordenar las tarjetas que representan mojarse las manos, enjabonarse y enjuagarse. El dibujo permite a los escolares identificar el paso; el tag permite al robot identificar la tarjeta y compararla con la secuencia configurada por el docente. Robotito reconoce los tags, sin interpretar el contenido de los dibujos u objetos.

Una vez ubicadas las tarjetas, se coloca a Robotito al inicio y se da comienzo al recorrido mediante las tarjetas de control. El robot sigue la línea y, al detectar el tag de una estación, se detiene y gira para leer el tag de la tarjeta colocada allí. Si coincide con el esperado para esa posición, enciende su anillo LED en verde; si no coincide, lo enciende en rojo. Tras mostrar el resultado, retoma el recorrido hacia la siguiente estación en ambos casos. Al llegar al final, si toda la secuencia fue correcta, realiza giros de festejo con el anillo encendido en verde; si hubo errores, se detiene y muestra los aciertos y errores de cada posición mediante los colores verde y rojo en su anillo LED. El grupo puede revisar el orden, reubicar las tarjetas y realizar un nuevo intento, con la mediación del docente.

Esta descripción sirve de referencia para el resto del informe. Los materiales y la dinámica de aula se detallan en el Capítulo 3, mientras que los estados del robot, las señales luminosas y los controles de pausa y reinicio se explican en la sección 3.3.3.

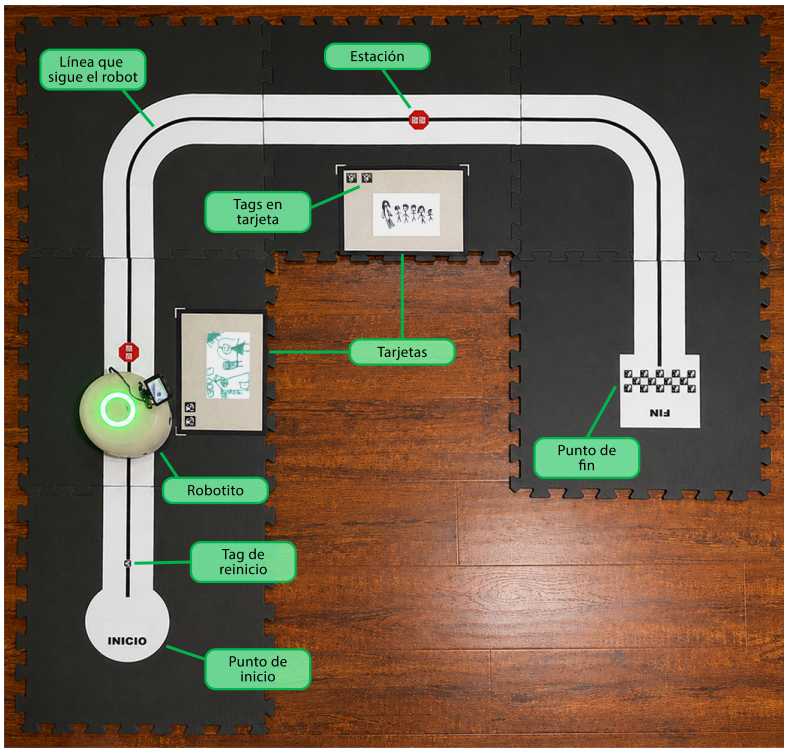


Figura 1.1: Montaje completo de la actividad. En la imagen se pueden observar el tablero modular, la línea que sigue Robotito durante el recorrido, el propio Robotito, los tags de referencia utilizados para detener o guiar la ejecución, las estaciones donde se ubican las tarjetas, y las tarjetas con dibujos colocadas por los estudiantes. También se observan el punto de inicio y el punto de fin del recorrido.

1.2. Objetivos

El objetivo general del proyecto es desarrollar una extensión de la plataforma Robotito que integre la cámara HuskyLens para ampliar sus capacidades de percepción visual y permitir que el robot procese la información obtenida y reaccione a ella. El alcance de la integración comprende el seguimiento de líneas y la lectura de tags, junto con un sistema de control basado en árboles de comportamiento (*Behavior Trees*) y la configuración inalámbrica desde una aplicación web.

Entre las actividades posibles que habilita esta extensión, se eligió diseñar e implementar la actividad didáctica de verificación de secuencias descrita en la sección 1.1, como caso de aplicación y validación de las capacidades incorporadas. Para alcanzar el objetivo general se determinaron los siguientes objetivos específicos:

- Implementar la comunicación entre la cámara HuskyLens y la placa SparkFun ESP32 Thing mediante el bus I2C.
- Traducir y adaptar el módulo de control de la cámara al entorno Lua RTOS de Robotito.
- Diseñar e implementar un módulo propio de *Behavior Trees*, que funcione como biblioteca de control general reutilizable para futuros proyectos.
- Integrar el sistema de percepción visual y el controlador BT dentro del flujo de tareas del robot.
- Desarrollar una actividad educativa basada en las capacidades de lectura de HuskyLens: seguimiento de línea, lectura de tags y validación de secuencias por parte del procesamiento del robot.
- Documentar la impresión de docentes y estudiantes sobre la actividad a partir de instancias piloto realizadas en un entorno similar al de una clase real.
- Extender el módulo existente de comunicación Bluetooth Low Energy (BLE) de Robotito para permitir la transferencia inalámbrica de configuraciones de actividad, con persistencia de datos en la memoria no volátil (NVS) de ESP32 Thing.
- Desarrollar un configurador web que permita registrar tags, definir secuencias de recorrido y enviarlas al robot sin necesidad de modificar el firmware.

Finalmente, la solución debe permitir separar con claridad la configuración de la actividad, la percepción visual y la lógica de control. Esto se ubicará en una biblioteca desarrollada que debe permitir que otros docentes e investigadores reutilicen los nodos de control en nuevos proyectos sin necesidad de modificar el código base. Cumpliendo con estos objetivos, se espera obtener una versión extendida de Robotito capaz de ejecutar una actividad configurable mediante un comportamiento autónomo organizado con árboles de comportamiento.

1.3. Organización del documento

El presente trabajo se estructura en seis capítulos y un apéndice técnico.

En el Capítulo 2 se presenta una revisión de antecedentes centrada en la plataforma Robotito y en los robots educativos existentes para actividades secuenciales, que sirvieron de referencia para definir el alcance de este proyecto.

En el Capítulo 3 se expone el análisis del problema y el diseño de la solución. Allí se retoma el problema abordado, se detallan los requerimientos identificados y se presenta el diseño general del sistema, incluyendo los módulos a implementar y la actividad educativa propuesta.

En el Capítulo 4 se describe la implementación de la solución, organizada en cuatro secciones. En primer lugar, se presenta el módulo de percepción visual con HuskyLens, detallando su funcionamiento y su integración con Robotito mediante el protocolo I2C. Luego se describe el control del comportamiento mediante Árboles de Comportamiento (*Behavior Trees*), incluyendo sus fundamentos teóricos y la arquitectura del módulo desarrollado. A continuación se presenta el configurador web utilizado para la preparación de secuencias, y finalmente se detalla la configuración de actividades y la comunicación BLE entre el configurador y el robot.

En el Capítulo 5 se documenta la experimentación realizada, tanto desde el punto de vista técnico como didáctico: entrevistas con docentes, sesiones prácticas con estudiantes, análisis de resultados y ajustes realizados a partir del feedback.

En el Capítulo 6 se presentan las conclusiones generales y las líneas de trabajo futuro.

Finalmente, el apéndice reúne material complementario. Las referencias bibliográficas se presentan al final del documento.

Capítulo 2

Revisión de antecedentes

2.1. Robotito

El robot Robotito fue desarrollado en la Facultad de Ingeniería de la Universidad de la República, pensado para introducir conceptos de robótica y pensamiento computacional en niños de entre 4 y 7 años. Se programa reorganizando los objetos de su entorno, sin necesidad de una interfaz de programación explícita. Previo a este proyecto, contaba con dos modalidades de interacción.

La primera modalidad utilizaba un sensor de color, ubicado en la base del robot, para detectar tarjetas de colores colocadas sobre la superficie de juego. Cada color indicaba una dirección de movimiento y el robot cambiaba su trayectoria al pasar sobre una tarjeta manteniendo el nuevo rumbo hasta encontrar otra. El estado activo se comunicaba mediante un anillo de LEDs RGB. El robot se activaba al apoyarlo en el piso y se detenía al levantarlo.

La segunda modalidad empleaba seis sensores de distancia distribuidos alrededor del robot para detectar objetos cercanos como conos. Robotito se desplazaba hacia el objeto más lejano detectado dentro de su alcance y si perdía de vista su objetivo realizaba un barrido de la zona para reencontrarlo o buscar uno nuevo.

2.1.1. Actividades

El equipo de Robotito documentó una serie de actividades organizadas de forma incremental en un libro destinado a docentes.

Las primeras actividades se centraban en la familiarización con el robot: conocer sus partes, aprender normas de uso y descubrir de forma exploratoria la relación entre las tarjetas de colores y el movimiento. A partir de allí, se introducían tareas de complejidad creciente, como resolver secuencias de tres pasos, planificar trayectorias evitando obstáculos representados con conos, o completar desafíos narrativos como ayudar al robot a recolectar frutas dispersas en la alfombra. Las actividades basadas en el sensor de distancia seguían una lógica similar, comenzando por la exploración libre del comportamiento del robot

y avanzando hacia ejercicios de predicción, donde los niños debían anticipar hacia qué objeto se movería el robot antes de encenderlo.

2.2. Robots educativos para actividades secuenciales

Existen varios robots educativos comerciales para niños de nivel inicial y primeros años de primaria que, como Robotito, se programan sin una interfaz de programación explícita. Algunos de ellos sirvieron de referencia para pensar la actividad de este proyecto. Bee-Bot (Hiper Escola, 2026) se controla mediante botones físicos sobre el propio robot. Estos permiten almacenar una secuencia de movimientos: avanzar, retroceder y girar. Cubetto (Primo Toys, 2026), en cambio, utiliza un tablero externo con bloques de colores insertables que codifican cada instrucción antes de ejecutar el recorrido, de forma similar a como Robotito se programa disponiendo tarjetas en su entorno.

Qobo (Robobloq, 2026) es un robot con forma de caracol pensado para niños de 3 a 8 años. El robot avanza sobre un recorrido armado con tarjetas tipo puzle y un sensor ubicado en su base lee la instrucción impresa en cada una a medida que las va recorriendo.

En todos estos casos la interacción se resuelve con sensores de contacto, lectura óptica de patrones impresos o botones, sin cámara que permita al robot interpretar su entorno.



(a) Bee-Bot (b) Cubetto (c) Qobo

Figura 2.1: Robots educativos con lógica de programación similar a Robotito.

Capítulo 3

Análisis del problema y diseño de la solución

3.1. Análisis del problema

Robotito forma parte de un proyecto de robótica educativa en constante crecimiento que ha recibido diversas modificaciones a lo largo del tiempo, con el principal objetivo de ampliar sus capacidades de programación y interacción.

Un ejemplo es el trabajo de Hitta (2022), en el que se desarrolló una interfaz de programación visual basada en Blockly. Esta interfaz permite que usuarios sin conocimientos de programación definan nuevos comportamientos para Robotito mediante una aplicación móvil. Posteriormente, Silveira Bonino (2024) desarrolló una interfaz de programación basada en tarjetas con tecnología RFID. Es una propuesta interesante que permite representar físicamente instrucciones y secuencias, facilitando el trabajo de conceptos de pensamiento computacional en actividades orientadas a la educación inicial.

De manera similar, este trabajo se enfoca particularmente en el proceso de integración de una herramienta de procesamiento de imágenes a Robotito, con el objetivo de extender su capacidad de relacionarse con su entorno y de esa manera permitir el planteo de actividades más complejas con materiales variados y más atractivos en un entorno infantil.

3.1.1. Problemas identificados

Originalmente la versión de Robotito sobre el que se trabajó contaba con un sensor en la parte inferior de su base capaz de reconocer colores y detectar cambios de distancia. Como se mencionó en el capítulo anterior, esto permitió la realización de actividades mediante la ubicación de tarjetas de colores sobre el piso, que luego eran detectadas por el robot al ubicarse este directamente arriba, para luego producir distintas reacciones en base a qué color era detectado por el sensor.

Si bien estas condiciones son suficientes para la realización diversas actividades con distintos niveles de dificultad, se pueden identificar algunas limitantes: cada tarjeta debe contener únicamente un color (evitando errores de lectura de color incorrecto); el robot debe estar posicionado directamente arriba de la tarjeta para poder identificar el color; si bien el sensor funciona en la mayoría de los casos, diferencias de iluminación en el entorno pueden generar inconsistencias de lectura.

Desde el punto de vista técnico, incorporar visión artificial en una plataforma embebida puede comprometer la estabilidad del sistema y afectar la capacidad de respuesta del robot. El procesamiento de imágenes requiere recursos significativos de CPU, memoria y tiempo de ejecución. En el caso de Robotito, el microcontrolador principal debe encargarse simultáneamente del control de motores, la lectura de sensores, la comunicación con otros módulos y la ejecución del entorno Lua RTOS. Además la solución a implementar debe ser estructurada de una manera que sea fácil de entender y mantener, con posibilidades de extender su funcionalidad a futuro.

3.1.2. Requerimientos de la solución

A partir de los problemas identificados se proponen siguientes requerimientos:

- **Bajo impacto computacional sobre el ESP32:** el procesamiento de imágenes no debe ejecutarse directamente en el microcontrolador principal. El sistema debe recibir información visual ya procesada, en forma de datos de alto nivel utilizables por la lógica de control.
- **Compatibilidad con la plataforma existente:** la solución debe integrarse con la arquitectura actual de Robotito, manteniendo el uso de la placa SparkFun ESP32 Thing, Lua RTOS y los mecanismos de comunicación disponibles.
- **Modularidad del software:** la percepción visual, la toma de decisiones y la ejecución de acciones deben mantenerse desacopladas, de forma de facilitar el mantenimiento, la extensión y la reutilización del sistema.
- **Reactividad ante eventos del entorno:** el robot debe ser capaz de responder a estímulos visuales durante la ejecución de una actividad, sin depender exclusivamente de secuencias rígidas de comandos.
- **Integración física robusta:** la incorporación de nuevos componentes no debe afectar negativamente la estabilidad, ergonomía ni facilidad de uso del robot. La cámara debe ubicarse en una posición adecuada para la detección visual y su montaje debía ser seguro y replicable.
- **Aplicabilidad didáctica:** la solución debe habilitar actividades comprensibles y manipulables por docentes y estudiantes, utilizando elementos más atractivos como parte activa de la experiencia educativa.

3.2. Diseño de la solución

Para cumplir con los requerimientos planteados se propone extender Robotito mediante una cámara HuskyLens, capaz de procesar imágenes y devolver información de alto nivel sobre los elementos detectados. De esta manera, el procesamiento visual no se realiza en el ESP32 y el robot puede reconocer elementos ubicados dentro del campo de visión de la cámara, sin depender únicamente del sensor situado en la parte inferior de su base.

La solución se organiza en módulos específicos que separan la percepción visual, el control del comportamiento y la configuración de la actividad. Esta organización permite integrar las nuevas capacidades con la arquitectura existente de Robotito, facilitar su mantenimiento y habilitar una actividad educativa configurable. A continuación se presentan los módulos necesarios para implementar la solución.

3.2.1. Módulos a implementar

La solución se organiza en cuatro módulos principales, cuyas responsabilidades se presentan a continuación. La implementación de cada uno se describe en detalle en el Capítulo 4.

En primer lugar, es necesario un módulo especializado que permita el intercambio de información entre HuskyLens y el robot. Este debe ser capaz de iniciar la comunicación con la cámara, construir mensajes y enviarlos, interpretar y validar sus respuestas, además de exponer los métodos necesarios que luego serán invocados desde el entorno de Lua RTOS.

Por otro lado, es necesario definir una capa de control organizada mediante el modelo de Árboles de Comportamiento, en la que cada nodo represente una acción, una condición o una estructura de control específica. Esta organización permite dividir el comportamiento general del robot en componentes simples y reutilizables, facilitando la incorporación de nuevas funcionalidades sin modificar por completo la lógica existente.

Finalmente, con el objetivo de facilitar la interacción del docente con Robotito y posibilitar la customización de la actividad a ser planteada, se decide implementar un configurador web en el que el usuario es capaz de editar la secuencia correcta esperada por el robot y enviársela utilizando el protocolo de comunicación BLE (Bluetooth Low Energy). Del lado del robot, es necesaria la implementación de un módulo capaz de realizar la conexión con el configurador web y transportar datos entre ambas partes, además de guardar correctamente los datos recibidos en el almacenamiento NVS (Non-Volatile Storage) de ESP32.

3.3. Diseño de la actividad

Una vez definidos los módulos a implementar se plantearon diversas actividades posibles, aplicables en entornos de educación inicial y acordes a la arquitectura actual de Robotito y las capacidades de percepción de HuskyLens.

Luego de establecer ciertos requisitos y realizar algunas pruebas, se logra definir un modelo de actividad.

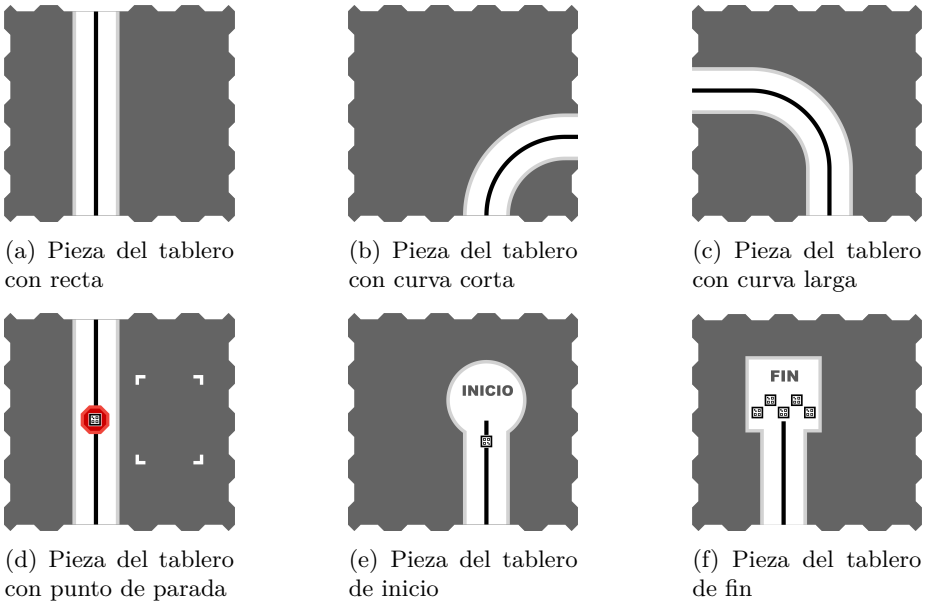
Una de las principales premisas establecidas es la de que el robot debe poder obtener información visual de su entorno y trasladarse según los elementos que haya identificado en cada lectura. Luego de realizar distintas pruebas con la cámara ya integrada al robot se determina que utilizar la función de Seguimiento de líneas (*Line Tracking*) es una manera confiable de cumplir con este requisito. Esto trae la necesidad de encontrar una forma de detener y reanudar el recorrido en determinados momentos específicos de la actividad. Entre las funciones disponibles de HuskyLens para cumplir con este requerimiento, se observa que los modos de Reconocimiento de color (*Color recognition*) y Reconocimiento de marcadores visuales (*Tag recognition*) son los que presentan mejores resultados prácticos. Si bien el modo de Reconocimiento de color puede resultar útil en algunas situaciones, se descarta esta opción al encontrar ciertas inconsistencias de lectura en casos de prueba con iluminación no favorable, en los que la cámara parece no detectar un color previamente identificado en otro caso con iluminación distinta del ambiente. Por esta razón, se concluye que la actividad debe contar con diversos marcadores visuales (de tipo *AprilTag*) a lo largo de un recorrido definido por una línea.

3.3.1. Caso de uso

Para desarrollar la actividad de verificación de secuencias descrita en la sección 1.1, se define un sistema que consiste en tres elementos principales (además de otros que se mencionan más adelante):

- Robotito con cámara HuskyLens integrada que recibe información visual del exterior y reacciona en base a los datos obtenidos.

- Un tablero que contiene un recorrido (definido por una línea contínua) y varios "puntos de parada" (la cantidad que sea necesaria, según la cantidad de elementos de la secuencia definida por el docente) que corresponden a cada posición de la secuencia a verificar por Robotito. Cada punto de parada se determina con un tag específico a lo largo del recorrido junto a una zona específica para ubicar las tarjetas. Además, el tablero contiene un punto de partida y uno de fin, donde Robotito debe empezar y terminar el recorrido, respectivamente.



(a) Pieza del tablero con recta

(b) Pieza del tablero con curva corta

(c) Pieza del tablero con curva larga

(d) Pieza del tablero con punto de parada

(e) Pieza del tablero de inicio

(f) Pieza del tablero de fin

Figura 3.1: Piezas del tablero

- Tarjetas correspondientes a cada elemento de la secuencia determinada por el docente. Cada tarjeta contiene un tag específico que luego será leído por Robotito en cada punto de parada, además de una sección libre donde el estudiante puede hacer dibujos, pegar una imagen o ubicar otro objeto arriba.



Figura 3.2: Ejemplo de tarjeta con tags

Cada pieza del tablero es un cuadrado encastrable de goma EVA de dimensiones 50cm x 50cm, con borde recto (sin encastre) en la zona donde se unen las líneas del recorrido (para evitar errores de lectura del robot), facilitando el armado y guardado por parte de docentes y alumnos. Además, la decisión de separar cada elemento del recorrido en distintas piezas agrega la posibilidad de crear recorridos distintos con curvas y rectas a lo largo del espacio en el que se utiliza.

Un ejemplo de recorrido completo se vería de la siguiente manera:

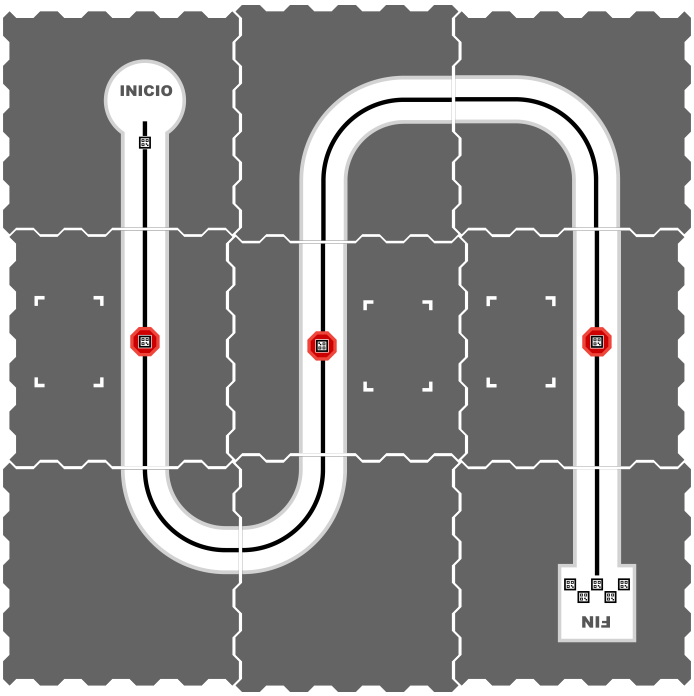
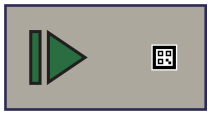


Figura 3.3: Ejemplo de recorrido completo

Además de estos materiales, la actividad cuenta con dos tarjetas adicionales, cada una con un tag específico a ser leído por el robot. El funcionamiento de cada una se explica en la sección 3.3.3 (Ejecución del robot).



(a) Tarjeta de Inicio



(b) Tarjeta para salir de modo de configuración

3.3.2. Desarrollo de la actividad

Definidos los materiales, se detalla la preparación y el desarrollo en el aula de la actividad presentada en la sección 1.1. En primer lugar, el docente debe configurar previamente la secuencia que será verificada por Robotito en el recorrido. Esto se traduce a definir una lista ordenada de tags a ser leída por Robotito durante el recorrido (un tag de cada tarjeta a ubicar en el tablero) y enviarla al robot. Este paso es explicado con más detalle en la sección 4.3, correspondiente al configurador web. En resúmen, en esta parte el docente define

qué tarjeta debe ir ubicada en qué lugar del recorrido.

Luego de configurada la actividad, en una instancia con alumnos el docente presenta el concepto de secuencia ordenada, poniendo como ejemplo secuencias que sean entendibles por niños de educación inicial, como las partes de un cuento conocido, las estaciones del año, pasos de la rutina diaria, o incluso una secuencia ordenada de números, entre otros.

Una vez presentados el concepto y ejemplos, se recomienda realizar una instancia de intercambio entre docente y alumnos en la que se presenta a Robotito y se habla brevemente sobre su aspecto, sus posibles funciones, relacionándolo con objetos de la vida cotidiana. De esta forma se puede presentar al grupo conceptos de robótica y pensamiento lógico, generando curiosidad e indagación. También se puede armar y hablar sobre el tablero, intentando llegar a un acuerdo sobre qué le parece al grupo que representa toda la actividad, así como mostrar las tarjetas y relacionar todo lo hablado con el concepto de secuencia ordenada.

Luego de presentados Robotito, el tablero y las tarjetas, y vinculado todo con el concepto de secuencia ordenada, el grupo debe ponerse de acuerdo en una secuencia específica a representar con las tarjetas, que luego serán ubicadas en el tablero y verificadas por el robot durante el recorrido. Para esto se pueden hacer dibujos (correspondientes a cada paso de la secuencia) en papeles que luego serán pegados en la zona libre de cada tarjeta, dibujos sobre la misma tarjeta, o incluso ubicar objetos arriba de cada una. El punto importante de este paso a tener en cuenta por el docente es que la relación entre el dibujo u objeto y la tarjeta debe ser correcta con respecto a la posición de cada uno en la secuencia. Por ejemplo, el dibujo correspondiente al primer paso de la secuencia debe estar pegado sobre la tarjeta que contenga el primer tag de la secuencia configurada, el objeto correspondiente al segundo paso de la secuencia debe estar ubicado sobre la tarjeta que contenga el segundo tag de la secuencia configurada, y así sucesivamente.

Una vez realizado el vínculo correcto por parte del docente entre tarjetas y pasos de la secuencia representados por los estudiantes, debe haber una instancia compartida en la que los estudiantes deben ubicar cada tarjeta (con su dibujo u objeto) en cada uno de los puntos de parada del recorrido armado.

Terminado el paso anterior, se ubica a Robotito en el punto de partida y se da comienzo al recorrido completo, en el que el robot se detiene en cada punto de parada para verificar si el tag de la tarjeta ubicada en ese punto corresponde a la posición del mismo en la secuencia. Si bien en este paso el robot está leyendo la información de los tags, al agregar un dibujo u objeto en cada tarjeta se genera la sensación de que el robot en realidad está mirando la imagen y no el tag (en el capítulo de Experimentación se detallan las actividades realizadas y las reacciones obtenidas). Luego de obtenida y procesada la información del tag correspondiente a la tarjeta en cada punto de parada, el robot reacciona con una retroalimentación luminosa dependiendo de si el elemento se encuentra en la posición correcta (luz verde) o incorrecta (luz roja) de la secuencia a verificar.

Finalmente, al llegar al punto de fin del tablero, el robot devuelve una retroalimentación general del recorrido, con el objetivo de informar si este fue

correcto o si hubo errores, indicando cuáles elementos leídos no se encontraban en la posición correcta. Esto permite que el recorrido se realice más de una vez, reubicando las tarjetas para indagar sobre cuál debería ser el comportamiento del robot en cada caso.

3.3.3. Ejecución del robot

Al encender Robotito, este se encuentra en modo "Configuración", con su anillo LED parpadeando en color azul. En este paso es posible conectarse al robot a través del protocolo BLE utilizando el configurador web y transmitir datos de secuencia esperada entre ambos. En caso de querer empezar el recorrido, el robot se debe ubicar en el punto de inicio del tablero apuntando la cámara hacia la línea del recorrido, y luego mostrarle (a la cámara) la tarjeta correspondiente a "salir de modo Configuración" (3.4b).

A partir de ahora Robotito se encuentra en modo "Esperando Inicio" (con su anillo LED parpadeando en color blanco). Para empezar el recorrido se debe mostrar la tarjeta de Inicio (3.4a), y el robot empezará a moverse siguiendo la línea.

Nota: Si durante el recorrido Robotito es levantado del tablero o pierde la línea que venía siguiendo, se pondrá automáticamente en modo "Pausa", con su anillo LED parpadeando en color amarillo. En este modo el robot recuerda la secuencia leída y verificada hasta el momento, por lo que solamente hay que reubicarlo en el lugar del tablero donde se perdió o levantó (con la cámara apuntando hacia la línea), y mostrar la tarjeta de Inicio para que este retome el recorrido.

Como se mencionó anteriormente, los puntos de parada se representan con un tag específico que el robot reconoce durante su avance.



Figura 3.5: Ejemplo de punto de parada

Se toma la decisión de representar estos puntos de parada de manera similar a una señal de tránsito "PARE" roja, como forma de hacer su significado más intuitivo, relacionable con el día a día, y agregar color al recorrido. Luego de realizar varias pruebas se agrega una segunda instancia del mismo tag dentro de la figura para evitar errores de lectura del robot, en caso que este no lea la primera.

Una vez leído el tag correspondiente al punto de parada, el robot detiene su

avance y empieza a rotar en sentido horario, hasta encontrar el tag de la tarjeta ubicada por el grupo en la zona específica. Con el objetivo de cumplir con que la tarjeta ubicada siempre se encuentre “a la derecha” del robot, se diseñan las piezas del tablero de manera que siempre se encastren de la misma forma, empezando con la pieza de “Inicio”, seguida por rectas y curvas que solamente se encastrarán dejando el área específica para ubicar la tarjeta del lado derecho del recorrido, finalizando con la pieza de “Fin”.

De igual manera que con los tags del punto de parada, se ubican dos tags iguales en la esquina de cada tarjeta (como se puede observar en 3.2), evitando errores de lectura del robot. Una vez leído alguno de los dos tags de la tarjeta, Robotito “pensará” brevemente (modo representado por su anillo LED prendiéndose progresivamente con luz blanca hasta completar el anillo, también pensado para darle al grupo una representación visual de lo que está haciendo el robot que, en este caso, significa la validación en tiempo real de la secuencia) y decidirá si la tarjeta leída se encuentra en la posición correcta o no. Si la posición es la tarjeta es correcta, el anillo LED se encenderá completamente con luz verde. En el caso incorrecto, se encenderá con luz roja.

El feedback luminoso se mantiene por un breve período de tiempo, luego se apaga y el robot empieza su movimiento de rotación, esta vez en sentido antihorario buscando nuevamente los tags del punto de parada. Una vez encontrado nuevamente el tag, Robotito sigue su recorrido por la línea del tablero.

Este proceso se repite la cantidad de veces que sea necesaria, según la cantidad de puntos de parada haya en el recorrido completo, finalmente llegando al punto de fin.

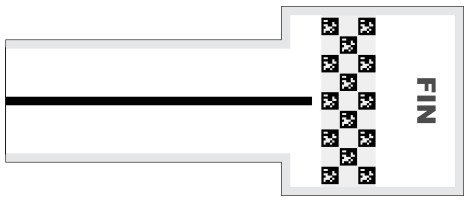
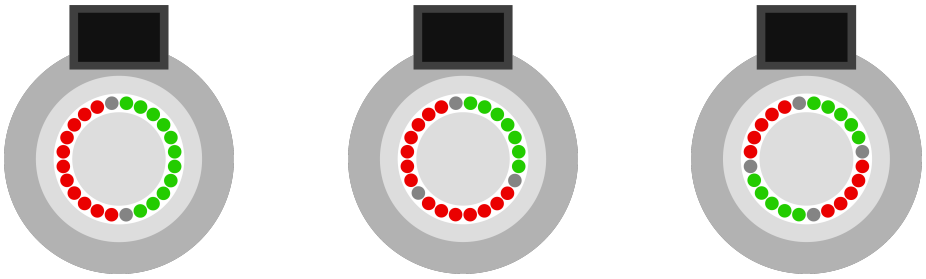


Figura 3.6: Punto de fin utilizado en las pruebas

Como se puede observar en la figura 3.6, el final del recorrido se representa por varias instancias del mismo tag formando una figura similar a una bandera de fin utilizada en carreras. Nuevamente, el objetivo es hacer la actividad intuitiva y relacionable con situaciones y símbolos del dia a día.

Al realizar la lectura de alguno de los tags ubicados en el punto de fin del recorrido, Robotito evalúa si todas las estaciones fueron visitadas en el orden correcto y retorna feedback de la verificación completa. En caso que la secuencia haya tenido errores de posición, Robotito se detiene y muestra en su anillo LED el resultado de cada posición en los colores rojo o verde, representando error o acierto, sucesivamente.



(a) Feedback final con un error, dos puntos de parada en total

(b) Feedback final con dos errores, tres puntos de parada en total

(c) Feedback final con dos errores, cuatro puntos de parada en total

Figura 3.7: Ejemplos de feedback final con error, con distintas cantidades de puntos de parada en el recorrido

En caso que la secuencia verificada sea correcta, Robotito sigue avanzando brevemente (pasando la zona representada por la "bandera final" de tags), y luego realiza giros en forma de "festejo" con su anillo LED encendido completamente en color verde.

Finalmente, en caso de querer reiniciar el recorrido, Robotito debe ser ubicado en el punto de Inicio del tablero, que además cuenta con un tag específico para reiniciar la actividad. Una vez leído ese tag, el historial de la secuencia validada hasta ese momento se descarta y el robot vuelve a modo "Esperando Inicio" (con su anillo LED parpadeando con luz blanca) listo para empezar un nuevo recorrido.

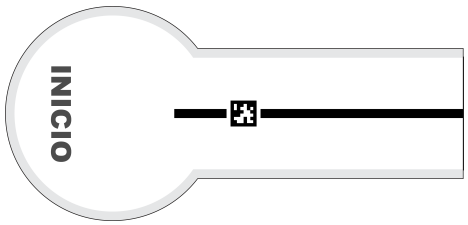


Figura 3.8: Punto de inicio utilizado en las pruebas (con tag de reinicio)

Capítulo 4

Implementación de la solución

4.1. Módulo de percepción visual con HuskyLens

Como se mencionó en capítulos anteriores, uno de los objetivos es incorporar una cámara a Robotito para ampliar sus funcionalidades. Para seleccionar la opción más adecuada, se realizó un análisis de las cámaras disponibles en el mercado.

Las características principales que se buscan son la compatibilidad con la placa ESP32, la capacidad de detectar colores, objetos y líneas, y la disponibilidad de estos algoritmos implementados directamente en la cámara para reducir el tiempo de integración. Además, se considera importante que la cámara tenga un costo y un tamaño adecuados, este úlitmo con el objetivo para poder instalarla en la parte superior del robot.

A partir de estos criterios, se identificaron varias alternativas disponibles en el mercado, las cuales se describen en Anexo A. Luego de evaluar cada opción, se seleccionó la cámara HuskyLens, dado que cumple con todos los requisitos establecidos.

Antes de describir su integración con Robotito, se presenta a continuación el funcionamiento general de la cámara: los algoritmos de visión que ofrece y la forma en que representa la información que detecta.

4.1.1. Funcionamiento de HuskyLens

Algoritmos de visión

HuskyLens incorpora distintos algoritmos de visión artificial que permiten realizar diferentes tareas, como el reconocimiento de objetos, colores y rostros, el seguimiento de líneas y la detección de marcadores visuales. Cada algorit-

mo procesa las imágenes capturadas por la cámara y devuelve la información correspondiente a las detecciones realizadas.

En este trabajo se utilizaron únicamente dos de estos algoritmos:

- **Seguimiento de líneas (*Line Tracking*):** permite detectar una línea dentro del campo visual de la cámara y seguir su recorrido. Este algoritmo se utiliza para implementar el seguimiento de líneas por parte de Robotito.

- **Reconocimiento de marcadores visuales (*Tag Recognition*):** permite detectar marcadores de tipo *AprilTag*. Para cada tag detectado, la cámara devuelve un identificador único y su posición dentro de la imagen, lo que permite al robot reconocerlo y localizarlo.

La información devuelta por estos algoritmos se representa mediante dos tipos de estructuras, las cuales se describen a continuación.

Representación de las detecciones

La información generada por los algoritmos de HuskyLens se representa mediante dos tipos de estructuras: *bloques* (*BLOCK*) y *flechas* (*ARROW*). La estructura utilizada depende del algoritmo que se encuentre activo.

Los bloques representan un objeto detectado mediante un rectángulo. Esta estructura es utilizada por el algoritmo de reconocimiento de marcadores visuales empleado en este trabajo, así como por otros algoritmos de la cámara. Cada bloque almacena la posición del objeto dentro de la imagen, su ancho, su alto y el identificador asociado a la detección. La Tabla 4.1 presenta los campos que componen esta estructura.

Tabla 4.1: Estructura lógica de un bloque (*BLOCK*)

Campo	**Descripción**
`xCenter`	Coordenada X del centro del rectángulo de detección
`yCenter`	Coordenada Y del centro del rectángulo de detección
`width`	Ancho del rectángulo
`height`	Alto del rectángulo
`ID`	Identificador del objeto aprendido

Con esta información es posible conocer dónde se encuentra el objeto dentro del campo visual de la cámara y el tamaño que ocupa en la imagen.

Las flechas representan una línea mediante un punto de inicio y un punto final. Esta estructura es utilizada por el algoritmo de seguimiento de líneas para describir el recorrido detectado. Además de las coordenadas de ambos puntos, cada flecha incluye un identificador asociado a la detección. La Tabla 4.2 resume los campos que forman parte de esta estructura.

Tabla 4.2: Estructura lógica de una flecha (*ARROW*)

Campo	Descripción
`xOrigin`	Coordenada X del origen de la flecha
`yOrigin`	Coordenada Y del origen de la flecha
`xTarget`	Coordenada X del extremo de la flecha
`yTarget`	Coordenada Y del extremo de la flecha
`ID`	Identificador del vector o trayectoria

Una vez presentado el funcionamiento de HuskyLens, se describe a continuación su integración con Robotito: la conexión física entre ambos dispositivos y el módulo de software desarrollado para comunicarlos.

4.1.2. Integración con Robotito

Conexión mediante I2C

El primer paso para integrar HuskyLens a Robotito consiste en conectar la cámara a la placa ESP32. La cámara dispone de un conector de cuatro pines que puede operar en modo UART o I2C. En este proyecto se utiliza el protocolo I2C, ya que es el mismo que emplean otros periféricos del robot, como el sensor de proximidad.

Una ventaja de esta elección es que no fue necesario desarrollar una nueva implementación del protocolo I2C, sino que se reutilizó la infraestructura de software ya existente en el proyecto. En la Tabla 4.3 y en la Tabla 4.4 se presenta la asignación de los pines utilizados para establecer la conexión entre la cámara y la placa ESP32.

Tabla 4.3: Asignación de pines del HuskyLens en modo I2C

Pin	Etiqueta	Función	Descripción
1	T	SDA	Línea de datos I2C
2	R	SCL	Línea de reloj I2C
3	GND	–	Referencia de tierra
4	VCC	–	Alimentación de 3.3 V a 5.0 V

Tabla 4.4: Conexión HuskyLens–ESP32 mediante I2C

HuskyLens	ESP32	Descripción
SDA (T)	GPIO 21	Línea SDA del bus I2C
SCL (R)	GPIO 22	Línea SCL del bus I2C
VCC	5V	Alimentación del sensor
GND	GND	Tierra común

Con la cámara conectada, resta implementar el módulo de software que permite intercambiar información entre HuskyLens y Robotito.

Implementación del módulo

El siguiente paso consiste en implementar un módulo en C que permita el intercambio de información entre la cámara y el robot.

La implementación se basa en la biblioteca oficial de Arduino para HuskyLens (DFRobot, 2021), adaptada al entorno ESP32/FreeRTOS y a la capa de comunicación I2C del sistema. En esta sección se describe el funcionamiento del módulo y su implementación a alto nivel. Para una descripción más detallada, puede consultarse el repositorio de (Cardozo, Sanson, Zamora, y Verdier, 2026a), donde se encuentra el código completo.

Como se observa en la figura 4.1, la implementación del módulo se organizó en cuatro capas, cada una con una responsabilidad claramente definida. Esta separación permite desacoplar la lógi- ca de comunicación de la interfaz utilizada por la aplicación, facilitando el man- tenimiento del código y la incorporación de futuras modificaciones.

En la base se encuentra la **capa de protocolo de comunicación**, que implementa el protocolo utilizado por la cámara. Su función es construir los mensajes que serán enviados a HuskyLens y reconstruir las respuestas recibidas, verificando que la información esté completa y sea válida antes de entregarla al resto del sistema. Esta capa trabaja únicamente con el formato de los datos intercambiados, sin interpretar su significado; no realiza acceso directo al hardware.

Sobre esta se encuentra la **capa de interfaz de acceso en C,**, que constituye el núcleo del módulo y coordina a las otras dos partes. Es la responsable de inicializar la comunicación con la cámara, enviar las solicitudes correspondientes apoyándose en la Capa 1 para construir los mensajes, recibir las respuestas y, a partir de los datos que la Capa 1 reconstruye, interpretar su significado. Además, mantiene internamente los resultados de las detecciones para que puedan ser consultados posteriormente por la aplicación.

A continuación se encuentra la **capa de interfaz para Lua**, que proporciona la funcionalidad utilizada por los programas escritos en este lenguaje. Su objeti- vo es adaptar las funciones implementadas en C al entorno de ejecución de Lua empleado por Robotito, ofreciendo una interfaz sencilla y ocultando los detalles internos de la comunicación con la cámara.

Finalmente, en el nivel superior se encuentra la **capa de aplicación**, correspondiente a los programas Lua ejecutados en Robotito. Esta capa consume la interfaz expuesta por la capa de interfaz Lua para solicitar información a la cámara y consultar los resultados de las detecciones.

El funcionamiento general del módulo comienza cuando la capa de aplicación solicita información a la cámara mediante la interfaz disponible en Lua o directamente desde código C. La interfaz de acceso solicita a la capa de protocolo que arme el mensaje correspondiente y lo envía utilizando el controlador I2C del sistema. Una vez recibida la respuesta de HuskyLens, la interfaz de ac-

ceso la reconstruye con ayuda de la capa de protocolo, interpreta la información obtenida y actualiza su estado interno. Posteriormente, la capa de aplicación puede consultar estos resultados sin necesidad de interactuar directamente con el protocolo de comunicación. Es importante notar que la interfaz de acceso actúa como coordinadora entre la capa de protocolo y el controlador I2C: ambas nunca interactúan directamente entre sí.

Esta organización permite que cada capa se ocupe de una tarea específica: la capa de protocolo implementa el protocolo de comunicación, la interfaz de acceso coordina el intercambio de información con la cámara, la interfaz Lua adapta esa funcionalidad al entorno de ejecución de Robotito y la capa de aplicación consume dicha interfaz para resolver la lógica propia de cada actividad. Como resultado, se obtiene una implementación modular, sencilla de mantener y consistente con la arquitectura del resto del software de Robotito.

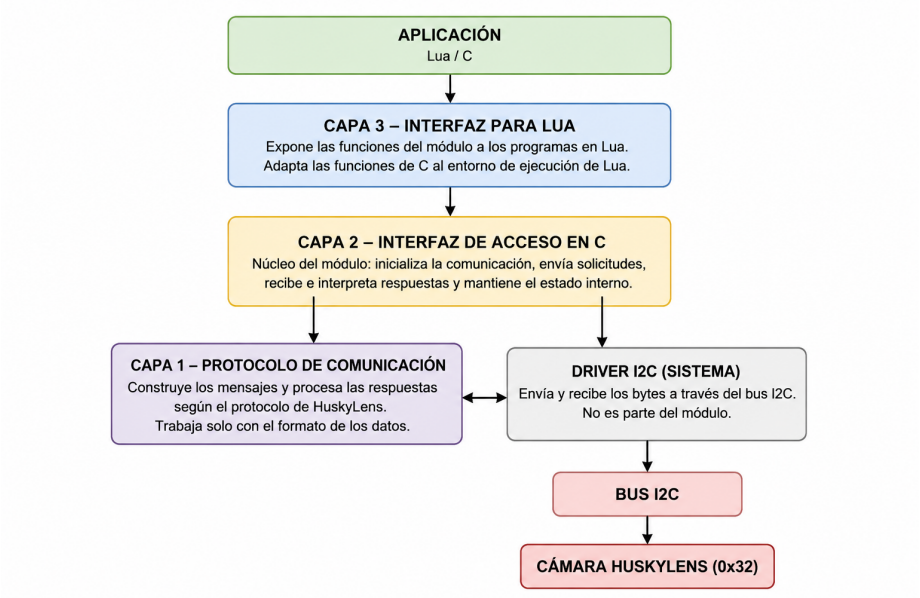


Figura 4.1: Arquitectura en capas del módulo de HuskyLens

4.2. Control del comportamiento con Behavior Trees

Otro de los objetivos es implementar un módulo de *behavior trees* en Lua, con el fin de contar con una alternativa al módulo de máquinas de estado que ya se utiliza en los scripts actuales de Robotito. Con este objetivo en mente, se implementó un módulo completo en Lua que permite desarrollar nuevos scripts para el robot utilizando árboles de comportamiento.

Antes de describir la implementación realizada, se presentan a continuación los fundamentos teóricos de esta arquitectura de control.

4.2.1. Fundamentos teóricos

Los Árboles de Comportamiento (*Behavior Trees*, BT) constituyen una arquitectura de control modular y jerárquica ampliamente utilizada en robótica moderna. Su principal ventaja es la capacidad de organizar comportamientos complejos mediante nodos bien definidos, manteniendo claridad, extensibilidad y una estructura altamente reutilizable.

Un BT se ejecuta mediante *ticks* periódicos. En cada tick se evalúan sus nodos desde la raíz, y cada nodo decide si retorna:

- **SUCCESS**: la tarea asignada se completó correctamente;
- **FAILURE**: la tarea no pudo completarse;
- **RUNNING**: la tarea continúa en progreso y deberá retomarse en ticks futuros.

Estos tres estados permiten que el robot reaccione rápido a los cambios del entorno, manteniendo un comportamiento continuo en el tiempo.

Tipos de nodos

En la teoría clásica de BT existen tres tipos principales de nodos:

- **Nodos de acción**: encapsulan operaciones concretas. Devuelven SUCCESS si la tarea se completa, FAILURE si es imposible finalizarla, y permanecen en RUNNING mientras la acción está en curso a través de múltiples ciclos.
- **Nodos de condición**: evalúan predicados instantáneos y retornan SUCCESS o FAILURE dependiendo de si la proposición se cumple o no.
- **Nodos compositores**: organizan el flujo del árbol y definen cómo se combinan los nodos hijos. Los dos compositores clásicos son:
  - *Sequence*: ejecuta a sus hijos de izquierda a derecha, avanzando al siguiente solo si el anterior devolvió SUCCESS. Si un hijo devuelve FAILURE o RUNNING, el nodo detiene la ejecución y propaga ese estado inmediatamente a su padre.
  - *Selector* (*Fallback*): diseñado para manejar alternativas o fallos. Intenta ejecutar a sus hijos en orden hasta que uno devuelve SUCCESS o RUNNING, y solo devuelve FAILURE si todos sus hijos fallan.

Con estos conceptos como base, se describe a continuación cómo se aplicó esta arquitectura en el módulo desarrollado para Robotito.

4.2.2. Arquitectura del módulo BT

Rol del BT dentro del sistema

En la arquitectura del robot, el BT se sitúa como una capa lógica que organiza y ejecuta decisiones de manera ordenada. El motor opera exclusivamente sobre un contexto que contiene el estado del sistema, y los nodos interactúan con funciones de bajo nivel a través de ese contexto. De esta forma, el BT no está atado a un conjunto fijo de acciones, sino que se encarga de coordinar el comportamiento general del robot.

Gracias a esta separación, el BT puede reutilizarse en distintas aplicaciones, no depende del hardware específico del robot, y permite modificar el comportamiento sin necesidad de cambiar su estructura interna. Esto facilita que el proyecto se pueda hacer crecer y mantener de forma ordenada.

Estructura general del módulo

El módulo `bt` implementa únicamente los componentes esenciales para componer comportamientos complejos:

- Nodos compositores: `sequence` y `selector`;
- Nodos hoja: `action` y `condition`;
- Decoradores: `repeater`, `interrupt`, `interruptible`;
- Árbol principal: construido mediante `bt.create_tree`.

A continuación se describe brevemente cada uno de estos nodos.

4.2.3. Implementación de nodos

El módulo `bt` desarrollado para este proyecto incluye una colección de nodos diseñada específicamente para funcionar de manera eficiente en Lua-RTOS sobre ESP32. A diferencia de otros motores BT más complejos, esta implementación busca ser simple, consumir poca memoria y evitar referencias circulares.

- `action(fn)`: encapsula una acción potencialmente prolongada, ejecutando la función `fn(self, ctx)` y retornando SUCCESS, FAILURE o RUNNING según corresponda.
- `condition(fn)`: evalúa una condición instantánea sobre el contexto, retornando SUCCESS o FAILURE sin mantener estado persistente.
- `sequence(children)`: ejecuta a sus hijos en orden y solo avanza al siguiente si el anterior devolvió SUCCESS, deteniéndose y propagando el estado correspondiente ante un FAILURE o RUNNING.
- `selector(children)`: ejecuta a sus hijos en orden buscando la primera alternativa exitosa, deteniéndose ante el primer SUCCESS o RUNNING y devolviendo FAILURE solo si todos sus hijos fallan.

- `repeater(child, n)`: repite la ejecución de su nodo hijo `n` veces, o indefinidamente si `n = -1`, reiniciando al hijo entre cada repetición.
- `interrupt(condition, child)`: representa un evento prioritario. Evalúa una condición externa y, si se cumple, ejecuta el nodo hijo asociado a la interrupción.
- `interruptible(child, interrupts, reset_on_interrupt)`: envuelve a un nodo hijo principal con una lista de nodos de interrupción, permitiendo abortar su ejecución de forma preventiva si alguno de ellos se activa.
- `create_tree(root)`: construye la estructura del árbol a partir de un nodo raíz, ejecutando `root:tick(ctx)` en cada invocación de `run()`.

Este conjunto de nodos conforma un motor BT completo y compacto, adecuado para sistemas embebidos. Mantiene la compatibilidad con las estructuras clásicas de *Behavior Trees* y suma mecanismos de interrupción pensados para entornos reactivos.

A modo de ejemplo, la figura 4.2 muestra el árbol de comportamiento implementado en Robotito, construido a partir de los nodos descritos en esta sección. En él puede observarse cómo los nodos compositores y decoradores se combinan para organizar las distintas tareas del robot en una estructura jerárquica y reactiva.

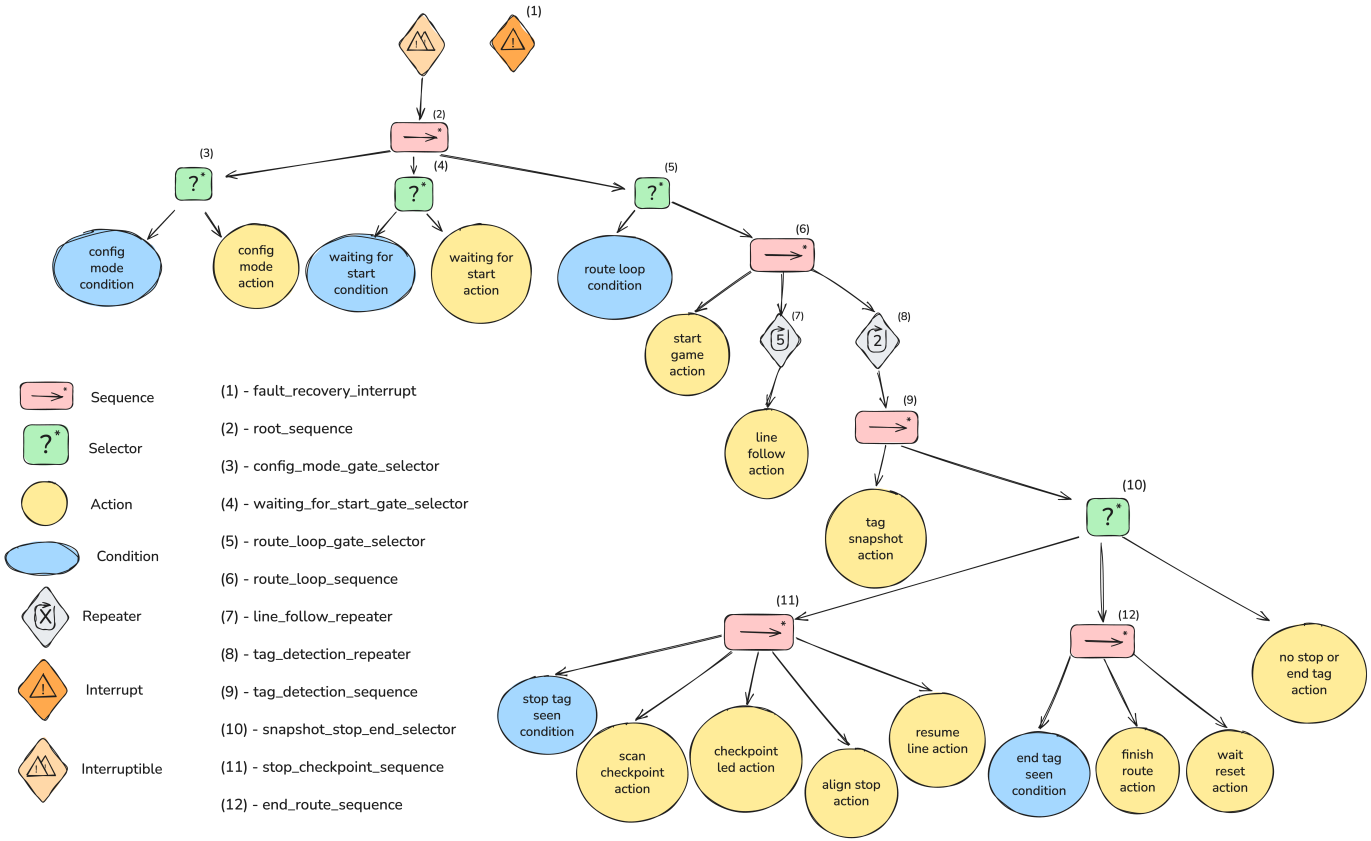


Figura 4.2: Árbol de comportamiento implementado en Robotito.

La implementación del modulo de BT se encuentra en el repositorio (Cardozo, Sanson, Zamora, y Verdier, 2026b).

4.3. Configurador web para la preparación de secuencias

Una vez definida la actividad como una secuencia de estaciones identificadas mediante tags, este capítulo presenta la herramienta web utilizada para preparar dicha secuencia antes de transferirla al robot.

4.3.1. Rol del configurador web

Se identificó la necesidad de contar con una herramienta que permitiera preparar recorridos y configuraciones de uso sin modificar directamente el firmware del robot ni depender de entornos de programación complejos. Con este objetivo, se desarrolló un configurador web orientado a facilitar la carga, organización y envío de la secuencia o secuencias de tags a validar por robotito.

Su propósito principal es permitir que el usuario registre tags, defina relaciones de equivalencia entre ellos, construya secuencias de recorrido y valide la configuración antes de su transferencia al robot.

4.3.2. Definición de tags y estaciones

La herramienta permite registrar tags, asociarles nombres o alias y organizarlos según su función dentro de la actividad. De esta forma, el usuario puede trabajar con identificadores comprensibles sin depender únicamente de valores numéricos.

Además, el configurador incorpora un mecanismo de intercambiabilidad para declarar que distintos tags cumplen la misma función dentro de la actividad. Esto permite que varios tags representen una misma estación cuando la consigna lo requiere.

Ejemplos de secuencias definidas en el configurador

Para ilustrar el uso del configurador, se presentan dos ejemplos concretos de secuencias que pueden definirse desde la web.

1. **Secuencia sin intercambiabilidad.** Se define una secuencia de cuatro estaciones con un tag distinto en cada caso:
   - Semilla (T_1)
   - Planta pequena (T_2)
   - Arbol (T_3)
   - Arbol con frutas (T_4)

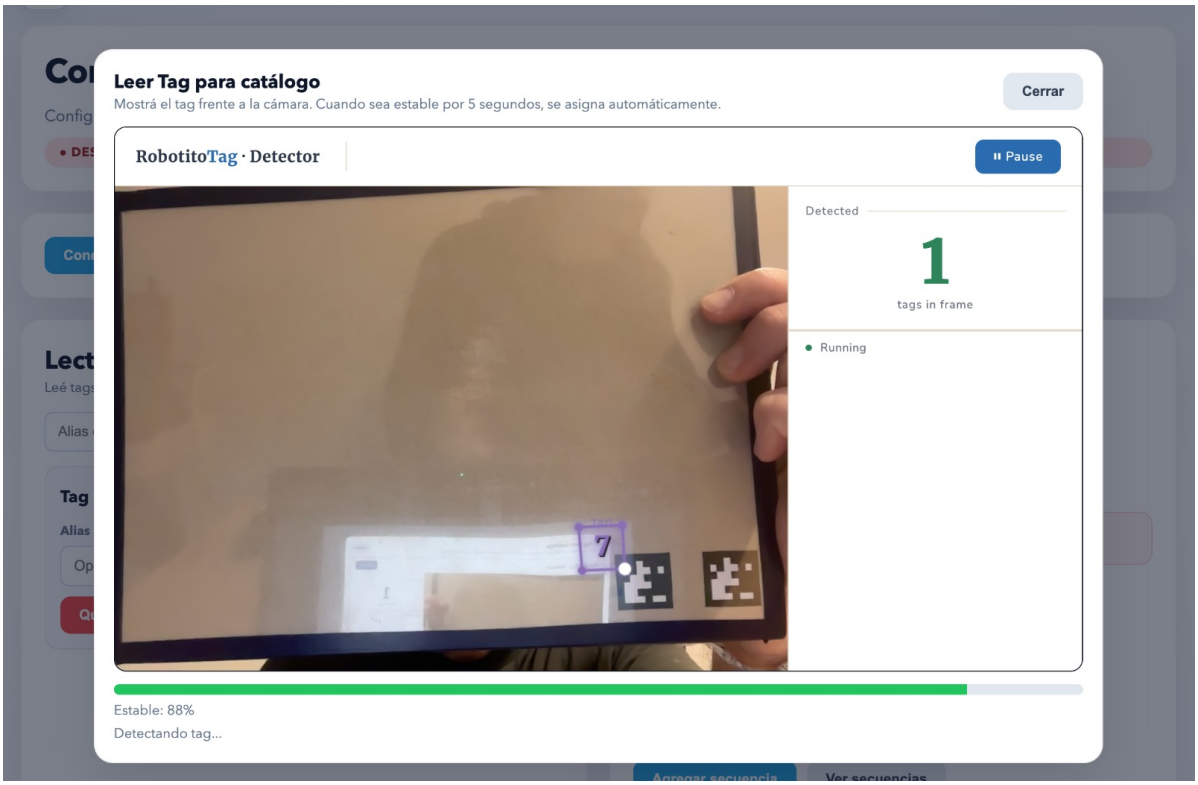


Figura 4.3: Lectura de tags.

En este ejemplo, el robot debe detectar exactamente el orden $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4$.

2. **Secuencia con intercambiabilidad.** Se propone formar correctamente la palabra *CASA*. La secuencia objetivo es:

   $$\text{C} - \text{A} - \text{S} - \text{A}$$

   Las letras A en las posiciones 2 y 4 se modelan con dos tags distintos (por ejemplo, A_2 y A_4), pero se declara una relación de intercambiabilidad entre ambos. De esta forma, el sistema acepta cualquiera de esos dos tags en ambas posiciones, manteniendo la validación de la palabra esperada.

4.3.3. Construcción y validación de secuencias

Una vez registrados los tags o cargada una actividad predefinida, el usuario puede construir una o más secuencias de recorrido. Cada secuencia representa el orden esperado de tags que el robot deberá validar durante la actividad.

Antes de enviar la configuración, el sistema verifica que exista al menos una secuencia definida, que no se superen los límites de tamaño de secuencia admitidos y que todos los tags incluidos se encuentren correctamente registrados. Estas validaciones buscan prevenir inconsistencias antes de transferir la información al robot.

4.3.4. Biblioteca de actividades y flujo de uso

El configurador incorpora una biblioteca de actividades predefinidas orientada a reducir el tiempo de preparación. A partir de ella, el usuario puede cargar configuraciones iniciales, ajustarlas según las condiciones del entorno y luego transferirlas al robot.

En términos funcionales, el flujo de uso del configurador es el siguiente:

1. El usuario selecciona una actividad predefinida o inicia una configuración manual.
2. El sistema incorpora los tags, grupos, estaciones y secuencias asociados a la actividad seleccionada.
3. El usuario registra o ajusta los tags disponibles.
4. El usuario define, si corresponde, qué tags pueden considerarse equivalentes.
5. Se construyen o modifican una o más secuencias de recorrido.
6. El configurador valida que las secuencias sean correctas.
7. La configuración queda pronta para ser transferida al robot.

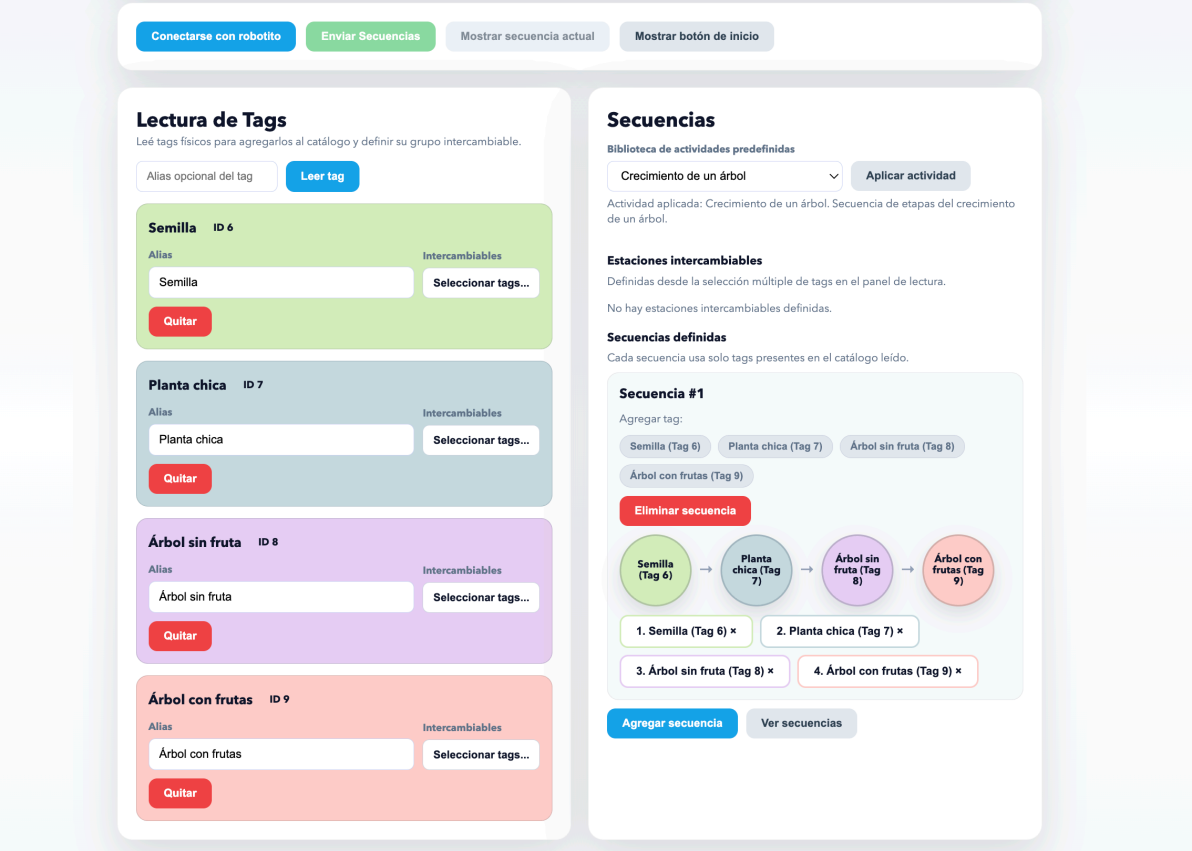


Figura 4.4: Gestión de secuencias.

4.4. Configuración de actividades y comunicación BLE

Luego de construir la configuración de la actividad en la aplicación web, es necesario transferirla al robot. Este capítulo describe la comunicación BLE utilizada para enviar y almacenar esa configuración.

El sistema requiere un mecanismo para transferir al robot la configuración definida por el docente sin modificar el firmware en cada sesión. Este capítulo describe la extensión del canal Bluetooth Low Energy (BLE) utilizada para enviar secuencias, almacenarlas en el robot y exponerlas a la lógica de ejecución.

Para dotar a Robotito de la flexibilidad necesaria en entornos educativos, se ha extendido la funcionalidad del módulo de comunicación *Bluetooth Low Energy* (BLE) original.

4.4.1. Gestión de Datos y Persistencia

La innovación central de la extensión es el *HuskyLens Mapper*, una capa lógica que permite desacoplar los identificadores físicos leídos por la cámara de la lógica semántica de la actividad.

Almacenamiento No Volátil (NVS)

Para que el robot sea usable en el aula sin configuraciones repetitivas, el módulo utiliza la memoria flash del ESP32 mediante el sistema *Non-Volatile Storage* (NVS). El robot restaura automáticamente estos datos al encenderse, permitiendo retomar secuencias previas de forma instantánea.

4.4.2. Arquitectura Multitarea y Sincronización

Para gestionar el flujo de datos proveniente del Configurador Web sin interferir con la ejecución de los Árboles de Comportamiento, se utiliza un mecanismo de **Ringbuffer** (buffer circular).

- **Recepción Asíncrona:** Cuando el stack de Bluetooth recibe una trama de configuración, la interrupción de hardware deposita los datos directamente en este buffer.
- **Tarea Dedicada:** Esta tarea de FreeRTOS permanece en estado de espera hasta que el buffer contiene información. Al activarse, extrae los datos y los prepara para la capa de aplicación, garantizando que la llegada de información no bloquee ni ralentice la locomoción o la percepción del robot.

4.4.3. Integridad de Datos y Mutex

Dado que el acceso a la memoria compartida donde se almacenan las secuencias puede ocurrir de forma concurrente, se han implementado primitivas

de sincronización llamadas **Mutex**.

Estos componentes actúan como semáforos de exclusión mutua: si la tarea de Bluetooth está actualizando o cargando datos desde la memoria **NVS**, el hilo de Lua tiene el acceso bloqueado hasta que la operación finalice. Este mecanismo es crítico para prevenir condiciones de carrera y asegurar que el Árbol de Comportamiento siempre valide la secuencia utilizando información íntegra y consistente.

4.4.4. Interfaz con la Capa de Aplicación (API Lua)

El módulo expone, a través de su API, las funcionalidades necesarias para conectar la lógica de aplicación con el canal de configuración inalámbrica. En la práctica, esta interfaz permite inicializar el stack de Bluetooth y definir el nombre de red visible para el configurador, preparar los mecanismos de sincronización y recuperar desde NVS las secuencias previamente almacenadas, registrar la función de Lua que se dispara automáticamente cuando el docente envía una nueva secuencia desde la web, persistir en memoria flash la lista de objetivos recibida para que el robot conserve la misión aprendida y, finalmente, entregar al Árbol de Comportamiento los identificadores lógicos que deben validarse en cada estación.

Para conocer en profunida la implementación del modulo de BLE, puede visitar el repositorio con su implementación completo (Cardozo y cols., 2026a)

Capítulo 5

Experimentación

La validación de plataformas educativas requiere no solo verificar su funcionamiento técnico, sino también evaluar su adecuación didáctica y usabilidad en contextos reales de enseñanza. En este capítulo se describe el proceso de experimentación llevado a cabo para validar la propuesta de extensión de Robotito, tanto desde la perspectiva docente como desde la experiencia directa con estudiantes. La experimentación se estructuró en dos etapas complementarias: primero, entrevistas semiestructuradas con docentes de nivel inicial y primaria para obtener retroalimentación sobre el diseño de las actividades y la viabilidad práctica de la propuesta; segundo, sesiones presenciales con grupos de estudiantes de educación inicial (entre 4 y 5 años de edad) y educación primaria (entre 7 y 8 años de edad) para observar la interacción directa con el robot y validar los supuestos de diseño.

5.1. Metodología

5.1.1. Entrevistas con docentes

Se realizaron entrevistas semiestructuradas con dos maestras de educación inicial y primaria: Elisa y Cecilia. Ambas docentes cuentan con experiencia en la integración de tecnología en el aula y habían trabajado previamente con robótica educativa en sus instituciones. Si bien no es necesario tener conocimientos tecnológicos o haber realizado algún tipo de actividad similar para poder utilizar Robotito como herramienta de enseñanza, fue beneficioso que sí lo hayan hecho ya que pudieron brindar comentarios útiles en base a experiencias previas que aportaron valor a la versión final del robot y de las actividades definidas. Las entrevistas tuvieron una duración aproximada de 50 a 60 minutos cada una y siguieron un guión diseñado para abordar los siguientes aspectos:

1. **Contexto y materiales físicos**: Evaluación de la propuesta de usar piezas de tatami tipo puzzle para formar la pista del robot, la claridad de los elementos visuales (calles blancas con líneas negras, puntos de parada

rojos, similitud con calles y señales de tránsito) y la viabilidad de que los niños armen el circuito de forma autónoma.

2. **Sistema de tarjetas con tags**: Opinión sobre el uso de etiquetas de tipo *AprilTag* para que el robot pueda leer y validar secuencias, la necesidad de herramientas auxiliares para que los docentes identifiquen el orden correcto de las tarjetas, y estrategias para evitar que los niños memoricen las secuencias según los códigos asociados.

3. **Retroalimentación del robot**: Tipos de feedback considerados apropiados (visual, sonoro, cinético) ante aciertos y errores, estrategias para manejar la frustración y momento adecuado para entregar la retroalimentación (paso a paso vs. al final del recorrido).

4. **Dinámica de juego y motivación**: Duración ideal de las actividades, balance entre trabajo colaborativo e individual, tipos de contenido más atractivos (cuentos, palabras, desafíos motrices), y estrategias de cierre.

5. **Aplicabilidad curricular**: Posibles integraciones con contenidos de diferentes áreas (lenguaje, matemática, ciencias naturales, etc.) y adaptabilidad a distintos niveles educativos.

6. **Consideraciones prácticas**: Dificultades previstas en términos de tiempo, espacio físico, desarrollo motriz de los niños y nivel de atención esperado.

Las entrevistas se realizaron de forma remota mediante videollamada, con consentimiento explícito de las participantes para el uso de la información obtenida con fines de investigación educativa.

5.1.2. Primera sesión práctica con estudiantes

Posteriormente se organizó una sesión presencial en coordinación con la maestra Cecilia, quien facilitó el acceso a dos grupos de estudiantes de segundo año de educación primaria (niños entre aproximadamente siete y ocho años) con aproximadamente treinta alumnos en total. La actividad se realizó en el salón de clases de la institución educativa.

La sesión se estructuró en etapas progresivas, siguiendo las recomendaciones surgidas de las entrevistas:

1. **Discusión previa**: Conversación general sobre la tecnología y la robótica en el día a día.

2. **Presentación del robot**: Introducción de Robotito y demostración de sus capacidades básicas de movimiento.

3. **Exploración de materiales**: Manipulación de las piezas de tatami por parte de los estudiantes, observación de las líneas negras y los puntos de parada rojos, e interacción con las tarjetas con tags e imágenes.

4. **Deducciones**: Etapa de conversación grupal en el que los niños presentan ideas y sugerencias sobre el funcionamiento esperado del robot.
5. **Construcción del circuito**: En grupos colaborativos, los niños armaron circuitos simples de 3 a 4 puntos de parada.
6. **Explicación del sistema de validación**: Se presentó el concepto de secuencia correcta y el rol de los tags.
7. **Ejecución y retroalimentación**: Recorrido del robot por los circuitos armados por los estudiantes, validando las secuencias mediante la lectura de tags y proporcionando retroalimentación inmediata con luces LED (verde para aciertos, rojo para errores).
8. **Acercamiento final**: Momento de acercamiento por parte de los niños para hacer preguntas y ver el robot de cerca, una vez finalizada la actividad y desarmado el recorrido.

Durante la sesión se realizaron observaciones sistemáticas sobre:

- Nivel de comprensión de las instrucciones
- Grado de autonomía en el armado del circuito
- Capacidad de comprensión del sistema de validación por tags
- Nivel de atención y motivación sostenida
- Reacciones emocionales ante aciertos y errores del robot
- Dinámicas de colaboración entre pares



Figura 5.1: Escuela Nº 168



Figura 5.2: Escuela Nº 168

5.1.3. Segunda sesión práctica con estudiantes

En coordinación con la maestra Maria José se organizó otra instancia de actividad práctica, esta vez en otro jardín y con niños de educación inicial (entre cuatro y cinco años). La actividad se realizó en un salón neutro de la institución educativa, esta vez con un grupo reducido de aproximadamente diez alumnos.

La sesión se estructuró en etapas progresivas con algunas diferencias con respecto a la actividad anterior, principalmente debido a la diferencia de edad entre los grupos:

1. **Presentación del robot y discusión**: Introducción de Robotito y conversación sobre sus capacidades de movimiento.
2. **Exploración de materiales**: Manipulación de las piezas de tatami por parte de los estudiantes, observación de las líneas negras y los puntos de parada rojos.
3. **Construcción del circuito**: Armado de circuitos simples de 3 a 4 puntos de parada.
4. **Explicación del sistema de validación**: Se presentó el concepto de secuencia correcta y el rol de las tarjetas.
5. **Ejecución y retroalimentación**: Recorrido del robot por los circuitos armados por los estudiantes, validando las secuencias mediante la lectura de tags y proporcionando retroalimentación inmediata con luces LED (verde para aciertos, rojo para errores).
6. **Acercamiento final**: Momento de acercamiento por parte de los niños para hacer preguntas y ver el robot de cerca, una vez finalizada la actividad y desarmado el recorrido.

En esta instancia no se pone énfasis en el concepto y utilidad de los tags. Según comentarios de las maestras se decide mejor realizar la actividad teniendo como foco principal el manejo del robot y el concepto de secuencia correcta y secuencia errónea.



Figura 5.3: Colegio Richard Anderson



Figura 5.4: Colegio Richard Anderson

5.2. Resultados

5.2.1. Retroalimentación de las entrevistas docentes

Las entrevistas arrojaron comentarios sobre varios aspectos del diseño que se detallan a continuación.

Diseño de materiales y elementos visuales

Las docentes validaron positivamente la propuesta de usar piezas de tatami modulares como base física de la actividad. Consideraron que el formato de puzzle es familiar para los niños y facilita tanto el armado colaborativo como la reconfiguración del circuito para diferentes actividades. Sin embargo, sugirieron considerar la durabilidad de los materiales, especialmente de los pegotines de calle con línea negra adheridas, dado el uso intensivo esperado. Respecto a los puntos de parada, ambas docentes recomendaron reforzar la señal visual del círculo rojo con elementos adicionales que hagan más explícita la acción esperada. Propusieron agregar íconos complementarios como ojos, manos o señales de "PARE" que los niños puedan asociar inmediatamente con la acción de detenerse.

Sistema de tarjetas con tags

El diseño propuesto de tarjetas—con un tag pequeño para la validación técnica ubicado en una esquina y una imagen central grande para los niños—fue bien recibido. Las maestras destacaron que esta dualidad permite que el contenido educativo sea visualmente predominante, mientras que el aspecto técnico queda en segundo plano. Elisa y Cecilia plantearon la necesidad de contar con una herramienta auxiliar (aplicación móvil o sistema web) que les permita a los docentes identificar rápidamente qué tag corresponde a cada tarjeta (o qué posición en la secuencia) al momento de preparar las actividades. Sugirieron también implementar un sistema de codificación visual en los bordes de las tarjetas (colores o patrones) que facilite la organización del material sin depender exclusivamente de la tecnología. Para evitar que los niños memoricen las secuencias por la ubicación física de las tarjetas, propusieron implementar un sistema de rotación de tag, para que en distintas sesiones identifiquen distintos pasos de la secuencia propuesta.

Progresión didáctica y etapas diferenciadas

Una de las recomendaciones más consistentes fue la necesidad de presentar la actividad en etapas claramente diferenciadas, especialmente para niños más pequeños (3 a 5 años). Elisa propuso la siguiente secuencia de complejidad creciente:

1. **Etapa inicial**: Introducción al robot y seguimiento libre de líneas sin puntos de parada ni validación.

2. **Etapa intermedia**: Incorporación de puntos de parada rojos sin validación de secuencia (el robot simplemente se detiene).

3. **Etapa avanzada**: Introducción del sistema completo con validación de tags y retroalimentación.

Esta progresión permite que los niños comprendan gradualmente cada componente del sistema antes de enfrentarse a la actividad completa.

Retroalimentación del robot

Las docentes coincidieron en que la retroalimentación inmediata es fundamental para el aprendizaje, pero debe diseñarse cuidadosamente para no generar frustración excesiva. Recomendaron un sistema de retroalimentación visual mediante luces LED de colores:

- **Verde**: Imagen en la posición correcta dentro de la secuencia.
- **Rojo**: Imagen en una posición incorrecta dentro de la secuencia.

Para el manejo de errores, sugirieron evitar que el robot vuelva automáticamente al inicio tras cada error, ya que esto podría resultar frustrante. En su lugar, propusieron que el robot se detenga en el punto de error, emita la señal visual correspondiente y espere a que los niños corrijan la tarjeta antes de continuar. Esta sugerencia buscaba mantener el flujo de la actividad y convertir el error en una oportunidad de reflexión y corrección inmediata. En la ejecución implementada, descrita en la sección 3.3.3, Robotito señala el error y continúa hacia la siguiente estación; el docente puede utilizar la función de pausa para intervenir y el grupo puede reordenar las tarjetas antes de un nuevo intento. Al completar exitosamente toda la secuencia, las maestras sugirieron una celebración multimodal: combinación de luces verdes parpadeantes, sonido festivo y, si es posible, algún movimiento expresivo del robot (giro, baile simple).

Control del flujo de la actividad

Un aspecto crítico identificado durante las entrevistas fue la necesidad de que los docentes puedan controlar el flujo de ejecución de la actividad. Las maestras plantearon dos situaciones recurrentes en el aula que requerían funcionalidades específicas:

Función de pausa Los docentes necesitan poder detener el robot en cualquier momento del recorrido para realizar intervenciones didácticas oportunas. Situaciones típicas incluyen:

- Aprovechar un momento de error para generar una discusión grupal sobre la secuencia correcta
- Atender interrupciones externas (llegada de otro docente, situaciones de convivencia)

- Permitir que todos los niños del grupo puedan observar un momento específico del recorrido
- Dar tiempo a los niños para reflexionar sobre lo que el robot hará a continuación

Función de reinicio (reset) La posibilidad de volver a comenzar el recorrido desde cero sin necesidad de apagar y encender el robot resultó fundamental para la dinámica del aula. Esta funcionalidad es necesaria para:

- Permitir múltiples intentos de la misma secuencia sin interrupciones técnicas
- Facilitar que diferentes grupos de niños prueben sus propias configuraciones
- Corregir errores de configuración inicial sin perder tiempo en reinicios manuales
- Mantener el ritmo de la clase cuando se trabaja con actividades breves y repetitivas

Estas sugerencias evidenciaron que el robot no solo debe ejecutar secuencias de manera autónoma, sino también ofrecer puntos de control que permitan al docente mantener la gestión didáctica de la actividad.

Aplicaciones curriculares

Las docentes identificaron múltiples posibilidades de integración curricular. La plataforma permite trabajar con secuencias asociadas a diversas áreas: narrativas de cuentos tradicionales en lenguaje y literatura, procesos naturales como el ciclo de vida de una planta en ciencias naturales, secuencias numéricas y patrones en matemática, o recorridos espaciales que conecten diferentes contextos sociales. Particularmente valoraron la posibilidad de que los niños creen sus propias secuencias mediante dibujos, otorgando mayor sentido de autoría a la actividad.

Duración y organización de las actividades

Las maestras recomendaron limitar las secuencias a 3-5 puntos de parada para niños de nivel inicial y primeros años de primaria. Actividades más largas podrían provocar pérdida de atención y dificultad para mantener en mente la secuencia completa. En cuanto a la duración total de la sesión, sugirieron bloques de 20 a 30 minutos, incluyendo tiempo para el armado colaborativo del circuito, la ejecución y la reflexión posterior. Consideraron fundamental incluir un momento final de cierre donde los niños puedan verbalizar lo que hicieron y aprendieron. Respecto a la organización, propusieron dinámicas de trabajo en pequeños grupos (3-4 niños) donde se puedan asignar roles rotativos: diseñador del circuito, colocador de tarjetas, observador del robot, y registrador de

resultados. Esta distribución de roles fomenta la participación equitativa y el desarrollo de diferentes habilidades.

5.2.2. Observaciones de la primera sesión con estudiantes

La sesión presencial con los dos grupos de segundo año de primaria permitió validar varios de los supuestos de diseño y observar directamente la interacción de los niños con el sistema.

Comprensión del sistema

Los estudiantes comprendieron rápidamente el concepto de seguimiento de línea y la función de los puntos de parada rojos. La analogía con las señales de tránsito (semáforo rojo = detenerse) facilitó la comprensión inmediata del sistema.

Un aspecto interesante observado durante la actividad fue la tendencia de los niños a asimilar el robot, o algunas de sus partes, con características humanas. Por ejemplo, varios estudiantes interpretaron la cámara como los "ojos" del robot y las ruedas como sus "piernas". Estas asociaciones espontáneas facilitaron la construcción de una representación intuitiva del sistema, ya que permitieron vincular componentes técnicos con referencias corporales conocidas por los niños.

Esta forma de interpretación también permitió introducir, de manera implícita, la idea de modularización del robot. Al reconocer que distintas partes cumplen funciones específicas, como ver, moverse o detectar elementos del entorno, se vuelve más comprensible que el robot pueda extender sus capacidades mediante la incorporación de nuevos módulos. En este sentido, la cámara no se percibe únicamente como un accesorio agregado, sino como una funcionalidad adicional que amplía lo que Robotito puede hacer.

Por otro lado, el concepto de secuencia correcta fue inicialmente más abstracto. Algunos niños asumieron que cualquier orden de las tarjetas sería válido. Sin embargo, al presentar ejemplos concretos con narrativas conocidas, la lógica de la secuencia se volvió evidente.

La función de los tags como mecanismo de validación técnica no fue completamente comprendida por todos los niños, pero tampoco resultó necesario que lo comprendieran en profundidad. Para ellos, el robot "sabía" si la tarjeta era correcta, lo cual era suficiente para participar en la actividad.

Autonomía en el armado

Los estudiantes de segundo año demostraron capacidad para armar circuitos simples de forma relativamente autónoma. Trabajando en grupos de 3-4 niños, lograron conectar las piezas de tatami siguiendo instrucciones básicas. Hubo algunos casos de piezas colocadas en orientaciones incorrectas (línea discontinua, ángulos cerrados), pero con mínima intervención del adulto lograron identificar y corregir los errores. La colocación de las tarjetas sobre los puntos de parada

requirió mayor guía inicial. Algunos niños intentaron colocar múltiples tarjetas en un mismo punto o no alineaban correctamente la tarjeta para que el robot pudiera leer el tag. Tras una demostración y práctica, mejoraron significativamente en esta tarea.

Motivación y compromiso

El nivel de motivación fue consistentemente alto durante toda la sesión. Los niños mostraron gran entusiasmo al ver al robot en movimiento y mantuvieron la atención durante las explicaciones cuando estas estaban acompañadas de demostraciones prácticas.

Los niños celebraban espontáneamente cuando el robot completaba exitosamente una secuencia, lo que sugiere que la actividad genera un sentido genuino de logro.

Reacción ante errores

Las señales de error (luz roja) no generaron frustración significativa. En la mayoría de los casos, los niños reaccionaron con curiosidad, intentando identificar qué tarjeta estaba en el lugar incorrecto. La decisión de que el robot continuara el recorrido tras señalar un error, sin regresar automáticamente al inicio, permitió mantener el flujo de la actividad. La señal luminosa sirvió como referencia para discutir el orden de las tarjetas y preparar nuevos intentos.

Colaboración

Las dinámicas colaborativas funcionaron bien en general. Los niños negociaban espontáneamente quién colocaba cada tarjeta y se ayudaban mutuamente a identificar errores. Sin embargo, en algunos grupos surgieron conflictos menores por el control del material o diferencias de opinión sobre el orden correcto. La maestra Cecilia intervino en estos casos sugiriendo turnos rotativos y promoviendo el diálogo sobre las razones de cada propuesta, lo que resultó en una experiencia más enriquecedora.

5.2.3. Observaciones de la segunda sesión con estudiantes

La segunda sesión presencial, realizada con un grupo de educación inicial de entre cuatro y cinco años, permitió observar la interacción de niños más pequeños con la plataforma. A diferencia de la primera instancia, el grupo fue reducido, con aproximadamente diez estudiantes, lo que permitió una dinámica más controlada y un acompañamiento más cercano por parte de las maestras.

Comprensión del sistema

Debido a la edad de los niños, la comprensión inicial del concepto de robot fue menos precisa que en la primera sesión. Algunos estudiantes asociaron Robotito con una aspiradora robot, utilizando como referencia un objeto tecnológico

más familiar en su entorno cotidiano. Si bien estas asociaciones no describían correctamente el propósito del sistema, sí funcionaron como punto de partida para comprender que se trataba de un dispositivo capaz de desplazarse de forma autónoma.

Los niños lograron identificar progresivamente que el robot podía moverse sobre el circuito y detenerse ante ciertas señales. Las señales de parada rojas resultaron especialmente comprensibles, ya que varios estudiantes las relacionaron con la idea de detenerse o esperar. Asimismo, manifestaron curiosidad por la cámara incorporada al robot, realizando preguntas o comentarios sobre su posible función.

Rol de las docentes durante la actividad

En esta instancia, la actividad requirió una mediación docente más fuerte que en la sesión anterior. Las maestras guiaron de forma más directa la presentación del robot, el armado del circuito y la interpretación de lo que ocurría durante el recorrido. Esta intervención fue necesaria debido a la edad de los estudiantes y permitió mantener la atención del grupo, ordenar la participación y traducir las consignas a explicaciones más concretas.

A diferencia de la primera sesión, no se generaron instancias extensas de discusión o debate colectivo. Sin embargo, los niños sí participaron mediante comentarios breves, preguntas espontáneas e intentos de anticipar qué debía hacer el robot en cada paso. Esto evidenció que, aunque la actividad debió ser más guiada, seguía habilitando formas iniciales de razonamiento sobre movimiento, secuencia y respuesta del sistema.

Participación e interés

La respuesta general de los estudiantes fue muy positiva. Una gran mayoría del grupo se mostró interesada en el robot, observó con atención sus movimientos y participó activamente cuando se les preguntaba qué debía ocurrir a continuación. Varios niños realizaron aportes espontáneos sobre el recorrido, las señales de parada y la función de los elementos visibles del robot.

El tamaño reducido del grupo favoreció que los estudiantes pudieran acercarse, observar el robot con mayor detalle y expresar ideas durante la actividad. Si bien las intervenciones no siempre fueron técnicamente acertadas, sí demostraron curiosidad y disposición a interactuar con la propuesta.

Adecuación de la actividad para educación inicial

La sesión permitió comprobar que la propuesta también resulta atractiva para niños de educación inicial, aunque requiere ajustes en la forma de presentación y en el grado de acompañamiento docente. Para este rango etario, la actividad debe apoyarse principalmente en consignas concretas, observación directa del comportamiento del robot y participación guiada en cada etapa del recorrido.

En este sentido, la actividad funcionó adecuadamente cuando fue presentada de forma breve, visual y altamente mediada por las maestras. La experiencia mostró que los niños de cuatro y cinco años pueden comprender qué hace el robot, identificar situaciones simples como el movimiento y la detención ante señales, y participar en la toma de decisiones durante la actividad, aunque todavía no logren formular con precisión qué es un robot o explicar su funcionamiento de manera abstracta.

5.3. Análisis, discusión y ajustes

Los resultados de la experimentación validan la viabilidad tanto técnica como didáctica de la propuesta. La integración de la cámara HuskyLens con el sistema de tags funciona de manera confiable en condiciones reales de aula, y el diseño de la actividad resulta comprensible y motivador para los estudiantes del rango etario objetivo.

5.3.1. Fortalezas identificadas

Tangibilidad y manipulación física El uso de materiales físicos (piezas de tatami, tarjetas) favorece el aprendizaje activo y permite que niños con diferentes estilos de aprendizaje se involucren en la actividad. La posibilidad de reconfigurar el circuito otorga un sentido de agencia y creatividad.

Retroalimentación inmediata y visible El sistema de luces LED proporciona retroalimentación clara, inmediata y no verbal, lo cual es especialmente apropiado para niños pequeños. La retroalimentación visual evita la necesidad de mediación constante del adulto.

Adaptabilidad curricular La plataforma no está atada a un contenido específico, lo que permite a los docentes integrarla de forma flexible en diferentes áreas y niveles educativos. Esta versatilidad aumenta significativamente su potencial de uso sostenido.

Aprendizaje colaborativo El diseño natural de la actividad promueve el trabajo en equipo, la negociación y el diálogo, habilidades fundamentales en el desarrollo infantil.

Progresión de complejidad La posibilidad de utilizar el sistema en diferentes niveles de complejidad (desde seguimiento simple de línea hasta validación completa de secuencias) permite adaptarlo a distintos grupos etarios y niveles de desarrollo.

5.3.2. Desafíos y áreas de mejora

Curva de aprendizaje inicial Los estudiantes de segundo año de primaria con quienes se realizó la sesión práctica lograron comprender e integrar todos los elementos del sistema (armado del circuito, comprensión de secuencias y validación por tags) dentro de una misma sesión de clase. Sin embargo, para niños de edades más tempranas, es probable que sea necesario planificar sesiones introductorias progresivas que presenten cada componente por separado, tal como sugirieron las docentes entrevistadas.

Gestión de materiales El sistema requiere múltiples componentes (piezas de tatami, tarjetas, robot) que deben organizarse, almacenarse y mantenerse. Esto puede representar una carga logística para los docentes, especialmente si trabajan con grupos numerosos o tienen limitaciones de espacio físico.

Necesidad de control del flujo de actividad La experiencia inicial mostró que sin mecanismos de pausa y reinicio, los docentes iban a tener dificultades para gestionar el ritmo de la clase. La imposibilidad de detener el robot para hacer intervenciones didácticas o de reiniciar rápidamente para nuevos intentos se determinó como limitante para la flexibilidad de uso en el aula. Este feedback fue determinante para la incorporación de los controles de flujo mencionados.

Dependencia de elementos técnicos Aunque el sistema es robusto, depende del correcto funcionamiento de la cámara y de la lectura de tags. En la primera sesión con estudiantes se observó una falla de lectura asociada a las condiciones de iluminación del salón, dado que en ese momento el robot utilizaba reconocimiento de colores para identificar las zonas de parada. Esta situación motivó un ajuste en el funcionamiento del sistema: las acciones del recorrido dejaron de depender de la detección de colores y pasaron a ser activadas mediante la lectura de tags. A partir de este cambio, la lógica principal de la actividad quedó basada en el seguimiento de líneas y el reconocimiento de tags, reduciendo la dependencia respecto a la iluminación ambiente para la detección de los puntos de parada. De todos modos, tags dañados, mal alineados o mal ubicados pueden interrumpir la actividad, por lo que es importante desarrollar protocolos para el mantenimiento y la resolución rápida de este tipo de problemas.

Necesidad de herramientas auxiliares para docentes La solicitud de una herramienta que permita a los docentes identificar tags mediante sus teléfonos móviles es legítima y necesaria. Sin esta herramienta, la preparación de actividades se vuelve más compleja y propensa a errores.

Riesgo de memorización Sin un sistema de rotación de tags, los niños podrían memorizar posiciones físicas en lugar de razonar sobre secuencias lógicas. Esto disminuiría el valor educativo de la actividad.

5.3.3. Validación de decisiones de diseño

La experimentación validó varias decisiones de diseño tomadas durante el desarrollo:

1. **Uso de HuskyLens sobre otras cámaras**: La capacidad de reconocimiento confiable de tags en tiempo real demostró ser adecuada para el contexto educativo. La alternativa de procesamiento remoto habría agregado complejidad innecesaria.

2. **Eliminación del reconocimiento de colores en puntos de parada**: La primera instancia con estudiantes permitió identificar que la detección de colores podía verse afectada por las condiciones de iluminación del salón. A partir de esta observación, se decidió que las acciones del recorrido no fueran activadas por reconocimiento de colores, sino mediante la lectura de tags. Esta modificación fue validada en la segunda instancia, donde no se observaron problemas de lectura asociados a la iluminación de la sala.

3. **Retroalimentación paso a paso vs. final**: La retroalimentación inmediata en cada punto de parada resultó más efectiva didácticamente que esperar al final del recorrido (Marwan, Akram, Barnes, y Price, 2022). Permite correcciones oportunas y mantiene el foco de los niños.

4. **Sistema modular de piezas de tatami**: El formato de puzzle resultó intuitivo y práctico. La modularidad permite escalabilidad (circuitos más largos o complejos) sin requerir nuevos materiales fundamentalmente diferentes.

5. **Integración de árboles de comportamiento**: Aunque no fue visible para los usuarios finales, la arquitectura basada en Behavior Trees facilitó la implementación de la lógica de control y permitirá futuras extensiones del comportamiento del robot de forma ordenada y mantenible.

5.3.4. Ajustes realizados basados en el feedback

Como resultado de la experimentación, se implementaron y planificaron diversos ajustes:

Mejoras en señales visuales de parada Se reforzaron los círculos rojos con iconografía adicional (señal de "PARE" estilizada) para hacerlos más explícitos, especialmente para niños más pequeños.

Calibración de retroalimentación luminosa Se ajustó la intensidad y duración de las señales LED para que sean claramente visibles incluso en ambientes bien iluminados, pero sin resultar abrumadoras. Se define además que previo a realizar la retroalimentación luminosa el robot debe mostrarse en un estado "pensativo" (sin movimiento, con las luces LED "girando" por un segundo) para darle tiempo a los niños que entiendan exactamente qué está pasando en cada paso del recorrido.

Implementación de controles de flujo En respuesta directa al feedback docente, se incorporaron dos funcionalidades esenciales:

1. **Función de pausa**: Se implementó un mecanismo que permite al docente detener la ejecución en cualquier momento realizando la acción de levantar el robot. Cuando se activa la pausa:
   - El robot detiene su movimiento inmediatamente y se activa el modo "pausa"
   - Se mantiene el contexto de la actividad (no pierde el progreso)
   - Puede reanudarse desde el mismo punto al reubicarlo en el recorrido y mostrar el tag correspondiente a "iniciar"
   - Permite intervenciones didácticas sin interrumpir el flujo lógico de la secuencia
2. **Función de reset**: Se agregó una etiqueta específica (tag) que permite reiniciar el recorrido y comenzarlo desde cero. Esta función:
   - Reinicia el estado interno del robot (contador de secuencia, validaciones previas)
   - Posiciona virtualmente al robot al inicio del circuito
   - Limpia los registros de aciertos/errores anteriores
   - Permite comenzar una nueva ejecución rápidamente sin manipulación física del robot

Ambas funcionalidades se diseñaron para ser accesibles en base a acciones simples, permitiendo que los docentes mantengan el control didáctico de la actividad en tiempo real sin depender de conocimientos técnicos avanzados.

Documentación de progresión didáctica Se elaboró material de guía para docentes que describe explícitamente las tres etapas de complejidad creciente (seguimiento libre, paradas simples, validación completa) con ejemplos de actividades para cada nivel, incluyendo ahora recomendaciones sobre cuándo y cómo utilizar las funciones de pausa y reset.

5.4. Conclusiones del proceso de experimentación

La experimentación realizada demuestra que la extensión de Robotito con capacidades de visión artificial y validación de secuencias es técnicamente funcional y didácticamente valiosa. La plataforma logra integrar pensamiento computacional, razonamiento secuencial y contenidos curriculares diversos en una experiencia de aprendizaje tangible y motivadora. El proceso iterativo de diseño, desde las entrevistas con docentes expertas hasta la validación práctica con estudiantes, resultó fundamental para identificar y resolver aspectos que no habrían

sido evidentes desde una perspectiva puramente técnica. Las recomendaciones de las maestras sobre progresión didáctica, gestión de la frustración y adaptabilidad curricular enriquecieron significativamente el diseño final. Las sesiones con estudiantes confirmaron que la propuesta es accesible y atractiva para el público objetivo, aunque también evidenció la necesidad de herramientas de apoyo para los docentes y protocolos claros para el uso efectivo del sistema. En particular, la incorporación de las funciones de pausa y reset surgió como un elemento crítico para permitir que los docentes mantengan el control didáctico de la actividad sin sacrificar la autonomía del robot. En conjunto, la experimentación valida que Robotito extendido con visión artificial constituye una herramienta educativa viable que puede contribuir al desarrollo del pensamiento computacional y al aprendizaje de contenidos curriculares en edades tempranas, cumpliendo así con los objetivos educativos planteados en este proyecto.

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones

Este trabajo aporta una extensión concreta y acotada de la plataforma educativa Robotito, desarrollada sobre cuatro frentes: percepción visual limitada al seguimiento de línea y a la lectura de tags mediante la cámara HuskyLens, configuración inalámbrica de actividades mediante un configurador web y un módulo de comunicación Bluetooth con persistencia en NVS, control modular del comportamiento del robot mediante Árboles de Comportamiento e integración de estas tres piezas en una actividad educativa concreta, puesta a prueba en dos instancias con estudiantes de entre 4 y 8 años.

Desde el punto de vista técnico, se desarrollaron tres componentes reutilizables e independientes entre sí, que en conjunto sostienen estos cuatro frentes. El módulo HuskyLens abstrae la complejidad del protocolo de comunicación con la cámara, exponiendo una interfaz uniforme accesible desde C y desde Lua, acotada a las dos capacidades utilizadas en este trabajo: seguimiento de línea y lectura de tags. El motor de Árboles de Comportamiento provee una biblioteca de control general, desacoplada de cualquier lógica de dominio específica, que permite modelar nuevos comportamientos robóticos combinando los nodos existentes sin modificar el núcleo del sistema. La extensión del módulo Bluetooth, junto con el configurador web, resuelve la configuración inalámbrica de actividades: permite preparar una secuencia desde una aplicación externa y transferirla al robot sin modificar su firmware, con persistencia entre sesiones. La integración de estos tres componentes en la actividad implementada muestra que la arquitectura funciona de forma coordinada en un escenario similar al de una clase real, y es la instancia donde efectivamente se validó el aporte del proyecto.

Un resultado adicional del proceso de desarrollo fue la importancia de iterar sobre decisiones técnicas a partir de la puesta a prueba con usuarios reales, dentro de ese mismo frente de integración: la primera sesión con estudiantes

evidenció que la detección de puntos de parada basada en reconocimiento de colores era sensible a las condiciones de iluminación del aula, lo que llevó a rediseñar esa lógica para depender exclusivamente de la lectura de tags. Este cambio fue validado en la segunda sesión, donde no se observaron problemas de lectura asociados a la iluminación, y confirma que someter la solución a condiciones de uso reales permitió detectar y corregir un problema que no era evidente desde un análisis puramente técnico.

Desde el punto de vista didáctico, la experimentación con tres maestras de nivel inicial y primaria y dos sesiones con estudiantes de entre 4 y 8 años permitió observar indicios de accesibilidad y motivación en los grupos participantes, dentro del alcance acotado de la actividad propuesta. Los niños de segundo año de primaria comprendieron rápidamente el mecanismo de seguimiento de línea y el significado de los puntos de parada; el concepto de secuencia correcta resultó inicialmente más abstracto, pero se volvió comprensible al presentarlo mediante ejemplos concretos y narrativas conocidas. Los niños de nivel inicial, por su parte, requirieron una mediación docente más fuerte para seguir la dinámica de la actividad, aunque mostraron interés y participación activa durante toda la sesión. En ambas instancias el nivel de motivación fue alto, y el feedback inmediato en cada punto de parada resultó, según lo observado y lo señalado por las docentes, útil para sostener la dinámica de corrección durante la actividad.

Las entrevistas y sesiones prácticas permitieron además identificar necesidades que no formaban parte del diseño original, en particular la falta de mecanismos para que las y los docentes pausaran o reiniciaran el recorrido sin perder el estado de la actividad. A partir de ese feedback se incorporaron al sistema funciones de pausa y reset, dentro del mismo frente de integración con la actividad educativa.

En conjunto, este trabajo constituye una extensión concreta y acotada de Robotito en percepción visual, configuración inalámbrica, control modular y una actividad educativa que integra las tres capacidades anteriores. La experimentación realizada es una validación inicial, en dos instituciones y con grupos reducidos, de que esta extensión es técnica y didácticamente viable dentro de ese alcance. Su desempeño en otros contextos, edades e instituciones queda abierto, y se retoma a continuación como líneas de trabajo futuro que extienden naturalmente los cuatro frentes ya desarrollados.

6.2. Líneas de trabajo futuro

Las siguientes líneas se plantean como extensiones naturales del alcance actual del proyecto: cada una toma uno de los cuatro frentes ya desarrollados (percepción visual, configuración inalámbrica, control modular o integración educativa) y propone profundizarlo, sin implicar un cambio de enfoque respecto de lo ya construido.

Persistencia de modelos aprendidos en tarjeta SD (percepción visual). Los modelos entrenados en la HuskyLens para líneas y tags se almacenan en la memoria interna del dispositivo y pueden perderse si un usuario manipula

accidentalmente el menú de la cámara. Guardarlos en una tarjeta SD permitiría restaurarlos, eliminando la necesidad de reentrenar el sensor ante cada desconfiguración, sin modificar el alcance de percepción ya definido.

Bifurcaciones en el mapa (control modular). Incorporar intersecciones en el circuito de tatami, donde el robot deba elegir un camino según el resultado de la validación, es una extensión directa del motor de Árboles de Comportamiento ya desarrollado: implicaría agregar nuevos nodos de decisión sin modificar el núcleo del sistema, introduciendo nociones de condicionales y ramificación lógica de forma concreta y visible.

Retroalimentación sonora (integración educativa). Complementar el feedback visual de los LEDs con señales sonoras diferenciadas para aciertos y errores, dentro de la misma actividad ya implementada, enriquecería la experiencia especialmente en grupos numerosos o con niños de menor edad, donde captar y mantener la atención del grupo completo es más desafiante.

Evaluación a mayor escala (integración educativa). Las dos sesiones realizadas constituyeron una validación inicial con grupos acotados en dos instituciones específicas. Una evaluación con múltiples docentes, niveles educativos e instituciones distintas, manteniendo la misma actividad como base, permitiría obtener evidencia más robusta sobre el impacto didáctico real y detectar aspectos de diseño que no emergen en contextos controlados. Esta línea no busca demostrar que la solución escala a un uso masivo, sino ampliar de forma gradual la evidencia disponible sobre su viabilidad.

Referencias

Blender Foundation. (2025). *Blender.* https://www.blender.org. (Software de modelado y renderizado 3D, versión 5.0.1, Accedido: 2026-02)

Cardozo, F., Sanson, J., Zamora, L., y Verdier, N. (2026a). *Código fuente hardware del proyecto.* Descargado de https://github.com/naniverdier/Lua-RTOS-ESP32-husky/tree/robotito-huskylens (Accedido: 2026-06-06)

Cardozo, F., Sanson, J., Zamora, L., y Verdier, N. (2026b). *Código fuente software del proyecto.* Descargado de https://gitlab.fing.edu.uy/nadia.verdier/firmware-husky/-/tree/husky-tree-tags?ref_type=heads (Accedido: 2026-06-06)

DesignLabX, P. (2024). *Modelo 3d de case para huskylens.* https://www.printables.com/model/639502-huskylens-case. (Accessed: 2025-10-02)

DFRobot. (2021). *Huskylens – an easy-to-use ai camera.* Descargado de https://www.dfrobot.com/product-1922.html (Accessed 2026-05-19)

Facultad de Ingeniería, Universidad de la República. (2024). *Modelo 3d de robotito.* https://redata.anii.org.uy/dataset.xhtml?persistentId=doi:10.60895/redata/SRXZ42. (Accessed: 2025-10-02)

Hiper Escola. (2026). *¿qué es la bee-bot?* Descargado de https://www.hiperescola.com/es/que-es-bee-bot (Accedido: 2026-07-22)

Hitta, S. (2022). *Interfaz de configuración de robotito para usuarios no programadores* (Tesis de grado, Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay). Descargado de https://hdl.handle.net/20.500.12008/34299 (Instituto de Computación)

Marwan, S., Akram, B., Barnes, T., y Price, T. W. (2022). Adaptive immediate feedback for block-based programming: Design and evaluation. *IEEE Transactions on Learning Technologies*, *15*(3), 406–419. Descargado de https://isnap.csc.ncsu.edu/home/public/papers/Marwan2022TLT.pdf doi: 10.1109/TLT.2022.3180984

Primo Toys. (2026). *Cubetto: A wooden robot that teaches kids to code.* Descargado de https://www.primotoys.com/ (Accedido: 2026-07-22)

Robobloq. (2026). *Qobo: Screen-free coding robot for early learners.* Descargado de https://www.robobloq.com/product/qobo (Accedido: 2026-07-22)

Silveira Bonino, F. (2024). *Diseño de una interfaz tangible de programación para robotito utilizando tecnología rfid* (Tesis de grado, Facultad de Inge-

niería, Universidad de la República, Montevideo, Uruguay). Descargado de https://hdl.handle.net/20.500.12008/47172 (Instituto de Computación)

Tejera, G., Amorin, G., Seré, A., Capricho, N., Margenat, P., y Visca, J. (2019, diciembre). Robotito: Programming robots from preschool to undergraduate school level. En *2019 19th international conference on advanced robotics (icar)* (pp. 296–301). Belo Horizonte, Brazil: IEEE. Descargado de https://ieeexplore.ieee.org/document/8981608 (Accedido: 2026-07-31) doi: 10.1109/ICAR46387.2019.8981608

Anexo A

Tabla comparativa de cámaras

Característica	**ESP32-CAM**	**Pixy2**	**OpenMV Cam H7 R2**	**HuskyLens AI**
¿Ya disponible?	Sí	Sí	No	No
Procesamiento integrado	Básico (a programar)	Sí (dedicado a visión)	Sí (MicroPython)	Sí (IA preentrenada)
Reconocimiento de colores	Sí	Sí	Sí	Sí
Detección de flechas	Manual con código	Nativo	Con programación	Con entrenamiento
Entrenable fácilmente	No	Por color	Programable	IA, aprendizaje simple
Facilidad de uso	Media-alta (técnica)	Muy alta	Alta (IDE amigable)	Muy alta (interfaz física)
Interfaz con microcontrolador	UART / WiFi / ESP-NOW	UART / SPI / I2C	UART / SPI / I2C / USB	UART / I2C
Lenguaje de programación	C++ / Arduino	No requiere (PixyMon)	MicroPython	No requiere
Costo aproximado	Muy bajo (6–10 USD)	Medio-alto (60–80 USD)	Medio (80 USD)	Medio-alto (60–90 USD)
Ideal para…	Prototipos económicos	Robótica educativa	Proyectos personalizados	Educación y prototipado rápido

Tabla A.1: Tabla comparativa de módulos de cámara para visión artificial.

Anexo B

Modelado de carcasa para Robotito

Con el objetivo de obtener una versión final del robot más robusta, estética y reproducible a mayor escala, se propone el diseño de una carcasa específica para la integración de Robotito con la cámara HuskyLens. En los prototipos iniciales los módulos se fijaban mediante soportes pegados o atornillados, con cables expuestos y componentes de sujeción visibles, lo que no garantizaba estabilidad mecánica ni una configuración consistente para el correcto funcionamiento del robot. El nuevo modelo permite encapsular los componentes en una estructura única compuesta por la carcasa original de Robotito modificada y una tapa adicional, de modo que el ensamblado sea directo y siempre resulte en la misma disposición validada durante las pruebas.

El diseño se realizó en función de los requerimientos de la cámara HuskyLens, fijando su posición de forma rígida y definiendo un ángulo específico de inclinación respecto al piso, ya que estos parámetros resultaron críticos para el correcto funcionamiento de los algoritmos de visión. Además, se incorporaron orificios internos para el guiado del cableado, logrando que este quede completamente contenido dentro de la estructura. De esta manera se logra la protección de los componentes, la confiabilidad del sistema y la calidad visual del robot, resultando en un dispositivo final adaptado específicamente a la aplicación educativa.

B.1. Desarrollo del diseño de la carcasa

Para evitar modificaciones internas a la disposición de los componentes ya existentes en Robotito, todo el trabajo de rediseño se realizó en la parte externa de la carcasa original. Para esto se trabajó sobre un modelo trisimensional de la carcasa exterior original de Robotito (Facultad de Ingeniería, Universidad de la República, 2024), que consiste en una única pieza con dimensiones aproximadas de 16.56cm x 16.42cm x 6.15cm (largo x ancho x alto).



(a) Vista superior



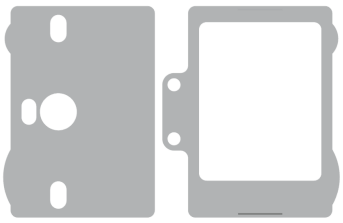
(b) Vista frontal



(c) Vista en perspectiva

Figura B.1: Vistas del modelo tridimensional de la carcasa original de Robotito utilizado como base para el rediseño.

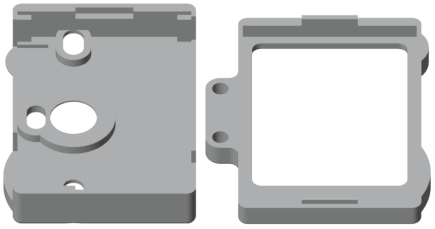
Además se buscaron modelos de soporte para HuskyLens, llegando a la decisión de utilizar un modelo de dos piezas (incluyendo una base y una tapa) de carcasa externa de la cámara (DesignLabX, 2024), con dimensiones aproximadas de 5.72cm x 4.82cm x 7.3cm para la base y 5.72cm x 4.12cm x 9.6cm para la tapa.



(a) Vista superior



(b) Vista Lateral



(c) Vista en perspectiva

Figura B.2: Vistas del modelo tridimensional de la carcasa original de HuskyLens.

B.1.1. Diseño conceptual

Con estos dos modelos como base para el diseño, se buscó fusionarlos en un solo modelo que permita agregar la cámara HuskyLens a su interior, manteniéndola fija en un ángulo determinado y de fácil acceso en caso de tener que configurarla o desarmar la estructura. Tomando como base las pruebas realizadas previas al rediseño de la carcasa, se definió el ángulo de inclinación de la cámara 16.5 grados con respecto al eje horizontal, que permite una visión cercana a Robotito, facilitando su funcionamiento, y que además logra mantener una coherencia estética y estructural con el modelo original, ya que un ángulo significativamente grande representa una pieza sobresaliente a la parte superior del robot que puede ser fácilmente rota al ser manipulada en un entorno educativo infantil.

B.1.2. Proceso de modelado CAD

La herramienta utilizada para el modelado tridimensional de la nueva carcasa fue Blender (Blender Foundation, 2025) en su versión para el sistema operativo

Windows. Con esta herramienta se logró importar los archivos de extensión .stl (Stereolithography) de cada modelo y trabajar sobre las superficies geométricas definidas en cada uno.

El primer paso para realizar el rediseño luego de obtenidos los dos modelos individuales de carcasa de Robotito y de cámara HuskyLens, fue ubicar a ambos en sus posiciones correspondientes al modelo objetivo. Para esto se ubica la carcasa de Robotito con el centro de su base en el origen (X=0, Y=0, Z=0) y luego se ubica el modelo de soporte de la cámara en la posición definida según las pruebas realizadas (ligeramente arriba de la parte superior de la carcasa de Robotito, sobre el borde del mismo, ubicando la zona correspondiente al lente de la cámara hacia la dirección "frontal" del robot, que además coincide con las ruedas frontales de Robotito, apuntando hacia "abajo" con una leve inclinación de 16.5 grados con respecto al plano horizontal).

Una vez ubicados ambos modelos en las posiciones correctas, se implementa una tercera estructura que respresenta la "unión" física entre ambos. Para eso se busca mantener la coherencia estética del modelo, intentando no utilizar ángulos muy forzados y simplificando la geometría utilizada.

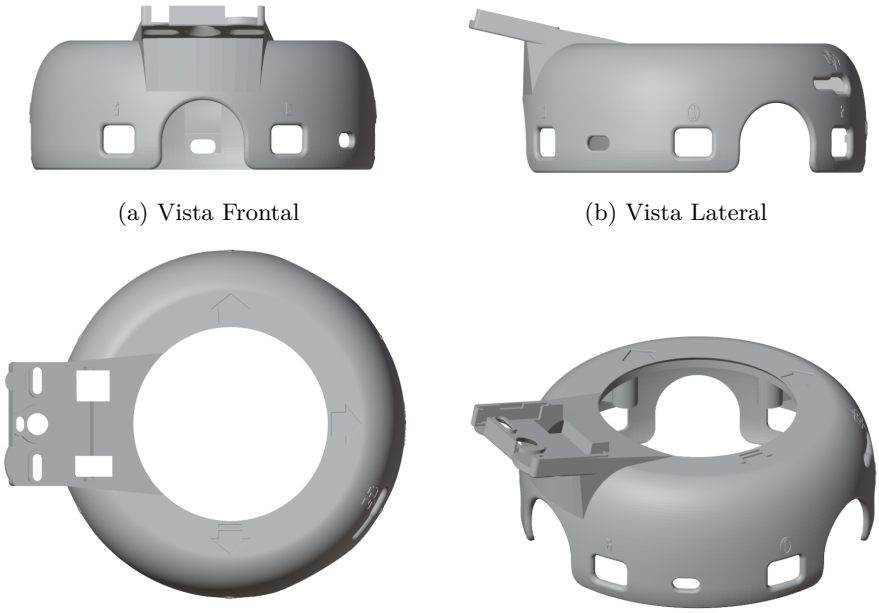
Con los tres modelos presentes y ubicados en sus posiciones finales, se realizan repetidas operaciones booleanas de Unión de Blender. Esto logra unir dos piezas seleccionadas en una sola, haciendo que el modelo consista de la menor cantidad de piezas necesarias. Luego de ejecutadas las uniones se llega a un modelo de dos piezas que incluye la carcasa original de Robotito con una base para HuskyLens, y una tapa para asegurar la cámara luego de ubicada en el robot.

Finalmente, para mejorar la estética del modelo y asegurar su funcionamiento seguro como aplicación en un entorno educativo infantil, se realizaron dos agujeros (en la herramienta Blender) en el modelo de la carcasa para permitir que todo el cableado de la cámara se ubique de forma interna a la estructura del robot. Cada agujero corresponde a una entrada física de cables que permite la cámara, con sus tamaños correspondientes.

B.1.3. Resultados y mejoras futuras

El resultado obtenido se detalla a continuación, donde se puede observar el diseño terminado con ambas piezas (carcasa y tapa) ya ubicados en sus respectivos lugares

Infelizmente no se pudo realizar la impresión del modelo por lo que este se mantiene conceptual. Resultaría interesante un rediseño del mismo para mejorar su aspecto estético y hacerlo más llamativo a un público objetivo infantil. A partir de esto se podría analizar si genera más interés en un formato de semejanza con lo "cotidiano", por ejemplo animales o vehículos, o quizás un formato más "robótico" que despierte curiosidad por su aspecto poco común.



(a) Vista Frontal

(b) Vista Lateral

(c) Vista Superior

(d) Vista en perspectiva

Figura B.3: Vistas del modelo tridimensional de la carcasa nueva de Robotito con soporte para HuskyLens.